NAAN MUDHALVAN PROJECT REPORT

FARM MANAGEMENT SYSTEM

A Salesforce-Based Agricultural Data Management Platform

Submitted by

S. Sayema

Register Number: [20624U18090]

Department of [COMPUTER SCIENCE]

[KAMBAN COLLEGE OF ARTS AND SCIENCE FOR WOMEN]

Academic Year 2026

TABLE OF CONTENTS

Chapter 1: Introduction

Chapter 2: Requirement Analysis

Chapter 3: Overview of the Salesforce Platform

Chapter 4: System Design

Chapter 5: Security Design

Chapter 6: Step-by-Step Development Guide

Chapter 7: Automation and Business Logic

Chapter 8: Apex Development

Chapter 9: Reports and Dashboards

Chapter 10: Testing

Chapter 11: Deployment and Maintenance

Chapter 12: Detailed Object Configuration and Screen Design

Chapter 13: Detailed Flow Build Guide

Chapter 14: Extended Apex and Lightning Web Component Development

Chapter 15: Experience Cloud Portal

Chapter 16: User Manual

Chapter 17: Project Plan, Data Migration and Extended Test Log

Chapter 18: Results, Future Scope and Conclusion

Appendices A to I and References

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Agriculture is the backbone of the Indian economy and supports the livelihood of more than half of the population. Despite its importance, a large share of farm data is still recorded manually in notebooks, on loose paper or in disconnected spreadsheets. Details such as land holdings, soil type, crop seasons, sowing and harvest dates, quantities harvested, buyer contacts and payment status are spread across many places. When a farm manager or a farmer needs a consolidated view, the information is either missing, outdated or inconsistent.

Cloud computing and low-code platforms make it possible to replace these scattered records with a single, secure and always-available system. Salesforce, the world's leading cloud platform, offers a declarative development model where objects, fields, relationships, automation and reports can be built with clicks rather than code, and where programmatic extensions in Apex are available for advanced needs. The Farm Management System (FMS) developed in this project uses these capabilities to digitize the complete agricultural workflow, from farmer registration to the final sale of the harvest to a buyer.

1.2 Project Overview

The Farm Management System is a Salesforce-based application that stores and manages information about farmers, farms, crops, harvests, buyers and orders. It provides role-based access so that system administrators, farm managers and farmers see only what they need. It automates repetitive tasks such as setting the initial crop stage, creating follow-up tasks, sending harvest reminders and updating delivery status when a payment is received. It also provides reports and dashboards that give real-time insight into production, sales and outstanding payments.

1.3 User Story

As a farmer, farm manager, or system administrator, I want to use a centralized Salesforce-based platform to record and manage farm, crop, farmer, and buyer information so that all agricultural data is stored securely, is easy to access, and supports better planning and decision-making.

1.4 Problem Statement

Traditional farm record keeping suffers from several problems that reduce productivity and income:

- Data is stored in physical notebooks or individual spreadsheets, which are easily lost, damaged or duplicated.
- There is no single view that connects a farmer to his farms, the crops on each farm, the harvest obtained and the buyers who purchased it.
- Important dates such as sowing, fertilizer application and harvest are tracked manually, so reminders are missed and yields suffer.
- Payment and delivery details of buyer orders are not tracked systematically, leading to disputes and delayed payments.
- Sensitive information such as identity numbers, pricing and income is not protected by any access control.
- Managers cannot generate summaries such as total acreage per region, crop distribution per season or revenue per buyer without hours of manual consolidation.

1.5 Need for the System

The need for a centralized farm management platform arises from the growing scale and complexity of modern farming. Farmers now cultivate multiple plots, grow different crops across Kharif, Rabi and Zaid seasons, and sell to multiple buyers including local traders, cooperatives and food-processing companies. Government schemes, insurance claims and bank loans also require accurate, verifiable records. A digital system brings accuracy, accountability and speed to all these activities.

1.6 Objectives of the Project

The main objectives of the Farm Management System are:

- To create a single source of truth for farmer, farm, crop, harvest, buyer and order data on the Salesforce platform.
- To model real-world relationships between these entities using lookup and master-detail relationships.

- To enforce data quality using validation rules, required fields, picklists and formula fields.
- To automate the crop lifecycle, alerting and order processing with Salesforce Flow and Apex.
- To secure data using profiles, roles, permission sets, organization-wide defaults, sharing rules and field-level security.
- To give decision makers real-time insight through reports and dashboards.
- To provide a scalable base that can be extended with a portal, mobile app, weather integration and AI-based prediction.

1.7 Scope of the Project

In scope: registration of farmers, farm and crop management, harvest recording, buyer management, order and payment tracking, automation, security, reports and dashboards, testing and documentation. Out of scope for the current version: real-time IoT sensor integration, payment gateway integration, GPS field mapping and machine-learning based yield prediction. These are listed as future enhancements in Chapter 18.

1.8 Target Users

- System Administrator: configures the org, users, security model, objects, automation and integrations.
- Farm Manager: supervises farms and crops within a region, records harvests, creates buyer orders and monitors dashboards.
- Farmer: views and updates the details of his own farms and crops and checks the status of orders for his harvest.
- Buyer: represented by a record in the system and linked to orders. Buyers receive email communication but do not log in during this version.

1.9 Methodology

The project follows an iterative, agile-style methodology. Requirements are captured as user stories, the data model is designed first, the application is configured declaratively, automation is added incrementally, and each increment is tested with sample data before the next is started. The phases are: requirement gathering, design, configuration and development, testing, deployment and review.

1.10 Organization of the Report

Chapter 1 introduces the project. Chapter 2 analyses requirements. Chapter 3 gives an overview of the Salesforce platform. Chapter 4 presents the system design and data dictionary. Chapter 5 describes the security model. Chapter 6 gives step-by-step development instructions. Chapter 7 covers automation with validation rules and flows. Chapter 8 contains Apex code. Chapter 9 describes reports and dashboards. Chapter 10 covers testing. Chapter 11 covers deployment and maintenance. Chapter 18 covers results, future scope and conclusion. Appendices provide formulas, SOQL queries, a glossary and viva questions.

CHAPTER 2: REQUIREMENT ANALYSIS

2.1 Requirement Gathering Approach

Requirements were collected by studying the typical activities of a small to medium farm business: how a farmer is onboarded, how land is recorded, how crops are planned per season, how harvest is measured and stored, how buyers are contacted and how payment is collected. Each activity was translated into a functional requirement and, where suitable, into a Salesforce feature.

2.2 Stakeholders

- Farmers: primary data owners who want a simple view of their land, crops and income.
- Farm managers or agricultural officers: responsible for planning and monitoring many farms.
- Buyers and traders: need clear order, quantity, price and delivery information.
- System administrators: responsible for maintaining the application and its security.
- Management or cooperative leadership: need summarized reports for strategic decisions.

2.3 Functional Requirements

ID	Requirement	Priority
FR-01	The system shall allow creation, editing and deactivation of farmer records with contact and identity details.	High
FR-02	The system shall allow a farmer to own multiple farms, each with location, area, soil type and irrigation type.	High
FR-03	The system shall allow multiple crops to be planned for each farm with season, variety, sowing date and expected harvest date.	High
FR-04	The system shall track the crop stage from Sowing to Harvested.	High
FR-05	The system shall record harvest quantity, quality grade and storage location for each crop.	High
FR-06	The system shall maintain buyer records with type and contact details.	High
FR-07	The system shall record orders linking buyer, harvest, quantity and price and calculate the total amount automatically.	High
FR-08	The system shall track payment status and delivery status of every order.	High
FR-09	The system shall prevent invalid data such as harvest date before sowing date or zero farm area.	High
FR-10	The system shall send an email alert to the farmer and manager seven days before the expected harvest date.	Medium
FR-11	The system shall create a follow-up task for the manager when a new crop is added.	Medium

ID	Requirement	Priority
FR-12	The system shall reduce available harvest stock when an order is placed.	Medium
FR-13	The system shall provide reports on crops, farms, yield, sales and payments.	High
FR-14	The system shall provide a dashboard summarizing key indicators.	High
FR-15	The system shall restrict data access based on user role and profile.	High

2.4 Non-Functional Requirements

- Security: all data is stored in the Salesforce cloud with encryption at rest and in transit, and access is controlled by profiles, roles and sharing rules.
- Availability: the application inherits the high availability of the Salesforce platform.
- Usability: the Lightning interface is responsive, uses guided flows for data entry and works on mobile browsers and the Salesforce mobile app.
- Scalability: data volume can grow to millions of records without redesign because of the multi-tenant architecture.
- Maintainability: configuration is declarative, well named and documented so that an administrator can modify it without code.
- Auditability: field history tracking records who changed important values such as payment status and quantity.

2.5 Use Cases

Use Case 1: Register a Farmer and Farm

Actor: Farm Manager. Precondition: user is logged in with the Farm Manager profile. Main flow: the manager opens the guided screen flow, enters farmer details, adds the first farm with area, soil and irrigation type, and submits. The system creates the Farmer and Farm records and links them. Postcondition: both records are visible in the Farmers and Farms tabs.

Use Case 2: Plan a Crop

Actor: Farm Manager or Farmer. The user opens a Farm record, clicks New in the Crops related list, selects the crop name, season and variety, and enters sowing and expected harvest dates. Validation ensures the dates are logical. On save, a flow sets the stage to Sowing and creates a follow-up task.

Use Case 3: Record a Harvest

Actor: Farm Manager. From the Crop record, the manager creates a Harvest with the date, quantity, grade and storage location. The Crop stage is updated to Harvested by a flow and the harvest becomes available for orders.

Use Case 4: Create and Track an Order

Actor: Farm Manager. The manager selects a Buyer and a Harvest, enters quantity and price per kg, and saves. The total amount is calculated. A trigger reduces the available quantity of the harvest. When payment is received, the manager changes Payment Status to Paid and the flow updates Delivery Status and notifies the buyer.

Use Case 5: View Dashboard

Actor: Farm Manager or Administrator. The user opens the Farm Management Dashboard to see crops by season, farms by location, expected versus actual yield and sales by buyer.

2.6 User Stories and Acceptance Criteria

ID	User Story	Acceptance Criteria
US-01	As a farm manager, I want to register farmers so that their details are stored centrally.	Farmer record saves with mandatory phone and name; duplicates are warned.
US-02	As a farmer, I want to see my farms and crops so that I can plan my work.	Farmer profile sees only own records.
US-03	As a manager, I want reminders before harvest so that labour and transport can be arranged.	Email sent exactly 7 days before expected harvest date.
US-04	As a manager, I want to record harvest so that stock is known.	Harvest requires quantity greater than zero.
US-05	As a manager, I want to create orders so that sales are tracked.	Total amount equals quantity multiplied by price.
US-06	As an admin, I want role-based security so that data is protected.	Users cannot view records outside their access.
US-07	As a manager, I want dashboards so that I can make decisions quickly.	Dashboard shows at least five components with live data.

2.7 Assumptions and Constraints

- A Salesforce Developer Edition or Trailhead Playground org is used, which limits storage and API calls but is sufficient for the project.
- Buyers do not log in; they are managed as records.

- Email deliverability depends on the org settings and is limited in Developer Edition.
- Currency is Indian Rupee (INR) and quantities are in kilograms.

2.8 Feasibility Study

- Technical feasibility: all requirements are achievable with standard Salesforce features and small amounts of Apex.
- Economic feasibility: the Developer Edition is free, so the development cost is limited to effort. A production deployment would need Salesforce licences.
- Operational feasibility: the Lightning UI is intuitive and needs minimal training for farmers and managers.
- Schedule feasibility: the declarative approach allows the full system to be delivered in a few weeks.

CHAPTER 3: OVERVIEW OF THE SALESFORCE PLATFORM

3.1 What is Salesforce

Salesforce is a cloud-based platform originally known for customer relationship management. Today it is a complete Platform as a Service (PaaS) and Software as a Service (SaaS) ecosystem where organizations can build custom applications without managing servers, databases or networks. Everything a user needs is delivered through a browser or the mobile app, and upgrades happen automatically three times a year.

3.2 Multi-Tenant Architecture

Salesforce uses a multi-tenant architecture in which many customers share the same infrastructure and application code while their data and customizations remain logically isolated. Each customer's configuration and data are stored as metadata and rows in shared tables, and governor limits ensure that no tenant can consume resources unfairly. For this project, this means the Farm Management System benefits from enterprise-grade security, backup and performance without any infrastructure effort.

3.3 Key Building Blocks Used in the Project

3.3.1 Objects and Fields

Objects are database tables and fields are columns. Standard objects such as Account, Contact and Task come with the platform, while custom objects, whose API names end with __c, are created for business-specific data. Field types used in this project include Text, Number, Currency, Date, Email, Phone, Picklist, Lookup, Master-Detail, Formula, Roll-up Summary, Checkbox and Text Area.

3.3.2 Relationships

A Lookup relationship is a loose link between two objects; the child can exist without the parent. A Master-Detail relationship is a tight link where the child inherits the sharing and security of the

parent, is deleted when the parent is deleted and supports roll-up summary fields on the parent. A Many-to-Many relationship is created using a junction object with two master-detail fields.

3.3.3 Lightning Experience

Lightning is the modern user interface of Salesforce. It uses apps, tabs, Lightning pages, record pages, compact layouts and components. App Builder lets an administrator drag and drop components to design a page. Path components guide a user through stages such as the crop lifecycle.

3.3.4 Salesforce Flow

Flow Builder is the declarative automation tool that replaced Workflow Rules and Process Builder. Record-triggered flows run when a record is created or updated, scheduled flows run at a defined time, screen flows present a guided user interface, and autolaunched flows are called from other automation.

3.3.5 Validation Rules

Validation rules verify data against a formula before a record is saved. If the formula evaluates to true, the save is blocked and an error message is shown. They are the first line of defence for data quality.

3.3.6 Apex and SOQL

Apex is a strongly typed, object-oriented language similar to Java that runs on the Salesforce servers. It is used for triggers, classes, batch jobs and integrations. SOQL (Salesforce Object Query Language) retrieves records and SOSL searches text across objects. Apex is subject to governor limits such as 100 SOQL queries and 150 DML statements per synchronous transaction, so code must be bulkified.

3.3.7 Security Model

Salesforce security is layered: organization-level (login hours, IP ranges, MFA), object-level (profiles and permission sets), field-level (field-level security), and record-level (organization-wide defaults, role hierarchy, sharing rules, manual sharing and teams).

3.3.8 Reports and Dashboards

Reports summarize data in tabular, summary, matrix or joined formats and can include filters, groupings and charts. Dashboards combine report charts into a single visual page with components such as bar, line, donut, gauge, metric and table, and can run as a specific user or as the logged-in user.

3.4 Why Salesforce for a Farm Management System

- Rapid development with clicks, not code, so the project can be completed quickly.
- Built-in security, audit trail and backup meet data protection needs.
- Mobile access allows farmers and field staff to use the system in the field.
- Rich automation, reporting and dashboard features are available out of the box.
- Easy extension through the AppExchange, Experience Cloud portals, APIs and AI features.

3.5 Comparison with Traditional Approaches

Aspect	Paper or Spreadsheet	Custom-Coded Web App	Salesforce FMS
Development time	None but limited	Months	Weeks
Security	None	Must be built	Built-in, granular
Automation	Manual	Must be coded	Flows and Apex
Reports	Manual	Must be coded	Drag-and-drop
Mobile access	Poor	Extra effort	Included
Maintenance	High effort	High effort	Low, declarative
Scalability	Poor	Depends on design	Very high

CHAPTER 4: SYSTEM DESIGN

4.1 Design Overview

The design of the Farm Management System is data centric. The core is a set of six custom objects that represent the real-world entities and their relationships. Around this core are a Lightning app for the user interface, a security layer, automation in the form of flows and Apex, and an analytics layer of reports and dashboards.

4.2 System Architecture

The architecture has five layers. The Presentation layer consists of the Lightning App, tabs, record pages and the mobile app. The Business Logic layer consists of validation rules, formula fields, flows and Apex classes and triggers. The Data layer consists of custom objects and relationships. The Security layer consists of profiles, roles, permission sets and sharing rules. The Analytics layer consists of reports and dashboards. All layers run on the Salesforce cloud platform.

4.3 Entity Relationship Description

- One Farmer can own many Farms (Farmer 1 : N Farm, lookup).
- One Farm can have many Crops (Farm 1 : N Crop, master-detail).
- One Crop can have many Harvest records (Crop 1 : N Harvest, lookup).
- One Buyer can place many Orders (Buyer 1 : N Order, lookup).
- One Harvest can be sold through many Orders (Harvest 1 : N Order, lookup).

This design means that from any Order it is possible to navigate upward to the buyer and the harvest, then to the crop, the farm and the farmer, giving complete traceability from field to buyer.

4.4 Data Dictionary

4.4.1 Farmer__c

Field Label	API Name	Type	Details
Farmer Name	Name	Text (80)	Standard name field, required
Phone	Phone__c	Phone	Required, 10 digits
Email	Email__c	Email	Optional, unique

Field Label	API Name	Type	Details
Address	Address__c	Text Area (255)	Village, district and state
Identity Number	Identity_Number__c	Text (20)	Encrypted or restricted by field-level security
Experience (Years)	Experience_Years__c	Number (2,0)	Years in farming
Status	Status__c	Picklist	Active, Inactive
Total Farms	Total_Farms__c	Roll-up or formula	Count of farms owned

4.4.2 Farm__c

Field Label	API Name	Type	Details
Farm Name	Name	Text (80)	Required
Farmer	Farmer__c	Lookup(Farmer__c)	Required
Location	Location__c	Text (100)	Village or survey number
District	District__c	Picklist	Used for regional reports
Area (Acres)	Area_Acres__c	Number (6,2)	Must be greater than zero
Soil Type	Soil_Type__c	Picklist	Red, Black, Alluvial, Laterite, Sandy, Clay
Irrigation Type	Irrigation_Type__c	Picklist	Drip, Canal, Borewell, Rainfed, Sprinkler
Status	Status__c	Picklist	Active, Fallow, Leased
Total Crops	Total_Crops__c	Roll-up Summary	Count of crops
Total Expected Yield	Total_Expected_Yield__c	Roll-up Summary	Sum of expected yield of crops

4.4.3 Crop__c

Field Label	API Name	Type	Details
Crop Record Number	Name	Auto Number	CROP-{0000}
Crop Name	Crop_Name__c	Picklist	Paddy, Wheat, Sugarcane, Cotton, Groundnut, Maize, Tomato, Banana and others
Farm	Farm__c	Master-Detail(Farm__c)	Required
Variety	Variety__c	Text (60)	Seed variety
Season	Season__c	Picklist	Kharif, Rabi, Zaid
Sowing Date	Sowing_Date__c	Date	Required
Expected Harvest Date	Expected_Harvest_Date__c	Date	Must be after sowing date

Field Label	API Name	Type	Details
Stage	Stage__c	Picklist	Sowing, Growing, Ready to Harvest, Harvested
Expected Yield (Kg)	Expected_Yield__c	Number (10,2)	Planning value
Days To Harvest	Days_To_Harvest__c	Formula (Number)	Expected date minus today
Notes	Notes__c	Long Text Area	Fertilizer, pesticide and remarks

4.4.4 Harvest__c

Field Label	API Name	Type	Details
Harvest Number	Name	Auto Number	HRV-{0000}
Crop	Crop__c	Lookup(Crop__c)	Required
Harvest Date	Harvest_Date__c	Date	Cannot be in the future
Quantity (Kg)	Quantity_Kg__c	Number (10,2)	Greater than zero
Available Quantity (Kg)	Available_Quantity__c	Number (10,2)	Reduced by orders
Quality Grade	Quality_Grade__c	Picklist	A, B, C
Storage Location	Storage_Location__c	Text (100)	Warehouse or godown

4.4.5 Buyer__c

Field Label	API Name	Type	Details
Buyer Name	Name	Text (80)	Required
Company	Company__c	Text (100)	Optional
Buyer Type	Buyer_Type__c	Picklist	Trader, Cooperative, Processor, Retailer, Exporter
Phone	Phone__c	Phone	Required
Email	Email__c	Email	Used for order notifications
Address	Address__c	Text Area (255)	Billing and delivery address

4.4.6 Order__c

Field Label	API Name	Type	Details
Order Number	Name	Auto Number	ORD-{0000}
Buyer	Buyer__c	Lookup(Buyer__c)	Required
Harvest	Harvest__c	Lookup(Harvest__c)	Required
Quantity (Kg)	Quantity_Kg__c	Number (10,2)	Cannot exceed available quantity
Price per Kg	Price_per_Kg__c	Currency (8,2)	INR
Total Amount	Total_Amount__c	Formula (Currency)	Quantity multiplied by price

Field Label	API Name	Type	Details
Order Date	Order_Date__c	Date	Defaults to today
Payment Status	Payment_Status__c	Picklist	Pending, Partial, Paid
Delivery Status	Delivery_Status__c	Picklist	Not Started, Ready, In Transit, Delivered

4.5 Tabs and Application Design

A Lightning app named Farm Management System is created with the navigation items Home, Farmers, Farms, Crops, Harvests, Buyers, Orders, Reports and Dashboards. Each object has a custom tab with its own icon. The app's utility bar shows Recent Items and Notes for quick access.

4.6 Page Layout and Record Page Design

- Farmer page: highlights panel with name, phone and status; related lists for Farms and Orders.
- Farm page: details section, related list of Crops with roll-up summaries, and a map or location field.
- Crop page: a Path component for Stage, a details section with dates and yield, related list of Harvests and activity timeline for tasks.
- Harvest page: quantity and available quantity, related list of Orders.
- Order page: buyer and harvest lookups, price, total amount and status fields grouped clearly.

4.7 Record Types and Business Processes

Record types can be added to Crop__c to distinguish Cereal, Vegetable and Cash Crop with different picklist values and page layouts. For example, cereals may show Grain Moisture while vegetables show Picking Frequency. This keeps the data entry screen relevant to each crop category.

4.8 Design Considerations

- Master-Detail between Farm and Crop ensures that crops are deleted with the farm and inherit its sharing.
- Lookup between Farmer and Farm avoids accidental deletion of farms when a farmer is removed.
- Picklists are used instead of free text to avoid spelling variation in reports.

- Auto number names give every crop, harvest and order a unique readable identifier.
- Field history tracking is enabled on Crop Stage, Order Payment Status and Harvest Available Quantity.

CHAPTER 5: SECURITY DESIGN

5.1 Security Objectives

Agricultural and financial data such as identity numbers, prices and payment status is sensitive. The security design follows the principle of least privilege: every user gets the minimum access needed to do the job, and access is widened only through explicit permission sets or sharing rules.

5.2 Layers of Security

Layer	Feature Used	Purpose
Organization	Password policy, MFA, login IP ranges, session timeout	Control who can log in and from where
Object	Profiles and permission sets	Control create, read, edit and delete on each object
Field	Field-level security	Hide identity number and pricing fields from unauthorized users
Record	OWD, role hierarchy, sharing rules	Control which records a user can see
Audit	Field history, setup audit trail	Track changes to data and configuration

5.3 Profiles

- System Administrator: full access to all objects, fields and setup.
- Farm Manager (custom, cloned from Standard User): create, read, edit on Farm, Crop, Harvest, Buyer and Order; read-only on Farmer identity number; no delete on Order.
- Farmer (custom, cloned from Standard User with minimum permissions): read and edit on own Farm and Crop; read on own Harvest and Order; no access to Buyer phone and email.

5.4 Roles and Hierarchy

The role hierarchy is: Administrator at the top, Regional Farm Manager below, and Farmer at the lowest level. Because Salesforce grants access upward in the hierarchy, a Regional Farm Manager automatically sees the records owned by farmers reporting to that role, without any additional sharing rule.

5.5 Organization-Wide Defaults

Object	Default Internal Access	Grant Access Using Hierarchies
Farmer__c	Private	Yes
Farm__c	Private	Yes
Crop__c	Controlled by Parent	Yes
Harvest__c	Private	Yes
Buyer__c	Public Read Only	Yes
Order__c	Private	Yes

5.6 Permission Sets

- Harvest Editor: adds create and edit access on Harvest__c to selected users.
- Order Manager: adds create, edit and delete on Order__c and read on Buyer__c contact details.
- Report Builder: allows creating and editing reports and dashboards.

Permission sets can be grouped into permission set groups so that a new manager can be given all required access in one step.

5.7 Field-Level Security

Identity_Number__c is visible only to System Administrator and Farm Manager. Price_per_Kg__c and Total_Amount__c are hidden from the Farmer profile when the farmer is not the owner of the harvest. Buyer phone and email are hidden from the Farmer profile.

5.8 Sharing Rules

A criteria-based sharing rule shares Farm__c records whose District__c equals a particular district with the public group of Farm Managers for that district. Owner-based rules share farms owned by the Farmer role with the Regional Farm Manager role in read and write mode.

5.9 Additional Protective Measures

- Enable multi-factor authentication for administrators and managers.
- Restrict login hours for the Farmer profile if required by the organization.
- Enable field history tracking and set up audit trail monitoring for configuration changes.
- Use Shield Platform Encryption in a production org for identity numbers.

CHAPTER 6: STEP-BY-STEP DEVELOPMENT GUIDE

6.1 Step 1: Create the Salesforce Developer Org

- Open developer.salesforce.com/signup and register for a free Developer Edition org using a unique username in email format.
- Verify the email, set the password and log in.
- Open Setup using the gear icon, search for My Domain and register a domain name such as fms-yourname.
- Deploy the domain to users and confirm that Lightning Experience is enabled.

6.2 Step 2: Create Custom Objects

- In Setup, type Object Manager in Quick Find, click Create, then Custom Object.
- For Farmer, enter Label Farmer, Plural Label Farmers, Record Name Farmer Name with Data Type Text.
- Tick Allow Reports, Allow Activities, Track Field History, Allow Search and Launch New Custom Tab Wizard.
- Save. Repeat for Farm, Crop, Harvest, Buyer and Order. For Crop, Harvest and Order set the Record Name data type to Auto Number with formats CROP-{0000}, HRV-{0000} and ORD-{0000}.

6.3 Step 3: Create Custom Fields

- Open each object, go to Fields and Relationships, and click New.
- Choose the data type from Chapter 4, enter the label, length, help text and whether it is required.
- For picklists, enter values one per line and, where appropriate, make the first value the default.
- For relationships, choose Lookup or Master-Detail as listed in the data dictionary and select the related object.
- Add each field to the page layout when prompted and grant field-level security to the required profiles.

6.3.1 Formula Field: Order Total Amount

Create a field of type Formula, data type Currency, named Total Amount, with the formula Quantity_Kg__c * Price_per_Kg__c. Set the blank field handling to treat blank fields as zeroes.

6.3.2 Formula Field: Days To Harvest

Create a Formula field of type Number with zero decimal places using the formula Expected_Harvest_Date__c - TODAY(). A positive number indicates days remaining and a negative number indicates a delayed harvest.

6.3.3 Roll-Up Summary Fields on Farm

- Total Crops: Roll-up Summary of type COUNT on Crop__c.
- Total Expected Yield: Roll-up Summary of type SUM on Crop__c.Expected_Yield__c.

Roll-up summary fields are available only because Crop is a master-detail child of Farm.

6.4 Step 4: Create Tabs

Go to Setup, Tabs, and under Custom Object Tabs click New for each object. Choose a tab style and icon such as a leaf for Farm, a wheat symbol for Crop, and a truck for Order. Make the tabs visible for the appropriate profiles.

6.5 Step 5: Create the Lightning App

- Setup, App Manager, New Lightning App.
- Name it Farm Management System and choose a logo or colour.
- Under Navigation Items add Home, Farmers, Farms, Crops, Harvests, Buyers, Orders, Reports and Dashboards.
- Assign the app to the System Administrator, Farm Manager and Farmer profiles.
- Save and open the app from the App Launcher to verify.

6.6 Step 6: Customize Page Layouts and Compact Layouts

Open each object in Object Manager, edit the page layout and arrange fields into sections such as Basic Information, Location, Dates and Status. Add related lists such as Farms on Farmer, Crops

on Farm, Harvests on Crop and Orders on Harvest. Create compact layouts showing the two or three most important fields so that the mobile highlight panel and hover cards are useful.

6.7 Step 7: Create the Crop Stage Path

- Setup, Path Settings, New Path.
- Choose object Crop and the picklist Stage, and give it the name Crop Lifecycle.
- For each stage add key fields and guidance for success, for example at Sowing show Sowing Date and Variety, and at Ready to Harvest show Expected Harvest Date and Expected Yield.
- Activate the path and add the Path component to the Crop record page.

6.8 Step 8: Create Profiles, Roles and Users

- Clone the Standard User profile twice and name the clones Farm Manager and Farmer.
- Configure object and field permissions for each as designed in Chapter 5.
- Create roles: Administrator, Regional Farm Manager and Farmer.
- Create at least three test users, one for each role, using different email addresses and assign profile, role and the Farm Management System app.

6.9 Step 9: Set Organization-Wide Defaults and Sharing Rules

Go to Setup, Sharing Settings, edit Organization-Wide Defaults and set the values from Section 5.5. Then create the sharing rules from Section 5.8 and recalculate sharing if prompted.

6.10 Step 10: Enable Field History Tracking

For each object, open Fields and Relationships, click Set History Tracking and select Crop Stage, Payment Status, Delivery Status, Quantity, Available Quantity and Farm Area. Add the History related list to the page layout.

6.11 Step 11: Load Sample Data

Create at least five farmers, eight farms, ten crops, six harvests, five buyers and ten orders. Data can be entered manually or imported using Data Import Wizard with CSV files. For import, prepare one CSV per object in the order Farmer, Farm, Crop, Harvest, Buyer, Order, and match lookups using record names or external IDs.

6.12 Sample Data Set

Farmer	Farm	Crop	Season	Area (Acres)	Expected Yield (Kg)
Murugan	Green Valley Farm	Paddy	Kharif	5.0	12000
Lakshmi	Sunrise Farm	Groundnut	Rabi	3.5	4500
Selvam	River Bend Farm	Sugarcane	Zaid	8.0	60000
Kavitha	Hill View Farm	Tomato	Rabi	2.0	15000
Ramesh	Palm Grove Farm	Banana	Kharif	4.5	30000
Anitha	Golden Field Farm	Maize	Kharif	6.0	18000
Suresh	Lake Side Farm	Cotton	Kharif	7.5	9000
Devi	Rainbow Farm	Wheat	Rabi	3.0	6000

CHAPTER 7: AUTOMATION AND BUSINESS LOGIC

7.1 Validation Rules

Validation rules ensure that only clean data is saved. Each rule has a name, an error condition formula, an error message and an error location.

Rule 1: Harvest Date After Sowing Date

Object: Crop__c. Formula: Expected_Harvest_Date__c <= Sowing_Date__c. Message: Expected harvest date must be later than the sowing date. Location: Expected Harvest Date field.

Rule 2: Farm Area Must Be Positive

Object: Farm__c. Formula: Area_Acres__c <= 0. Message: Farm area must be greater than zero.

Rule 3: Phone Number Format

Object: Farmer__c. Formula: NOT(REGEX(Phone__c, "[0-9]{10}")). Message: Enter a valid 10 digit phone number.

Rule 4: Harvest Quantity Positive

Object: Harvest__c. Formula: Quantity_Kg__c <= 0. Message: Harvest quantity must be greater than zero.

Rule 5: Harvest Date Not in Future

Object: Harvest__c. Formula: Harvest_Date__c > TODAY(). Message: Harvest date cannot be in the future.

Rule 6: Order Quantity Within Stock

Object: Order__c. Formula: Quantity_Kg__c > Harvest__r.Available_Quantity__c. Message: Order quantity exceeds the available harvest stock.

Rule 7: Price Must Be Positive

Object: Order__c. Formula: Price_per_Kg__c <= 0. Message: Price per kg must be greater than zero.

Rule 8: Delivered Requires Paid

Object: Order__c. Formula: AND(ISPICKVAL(Delivery_Status__c, "Delivered"), NOT(ISPICKVAL(Payment_Status__c, "Paid"))). Message: An order cannot be marked delivered until payment is complete.

7.2 Flow 1: Crop Creation Automation

Type: Record-triggered flow on Crop__c, runs after a record is created. Steps: the flow updates Stage__c to Sowing using an Update Records element, then creates a Task with Subject Review new crop plan, Due Date equal to Sowing Date plus 7 days, and assigns it to the farm owner or the manager. The flow ends after logging success.

7.3 Flow 2: Harvest Reminder

Type: Scheduled flow that runs daily. The Get Records element finds all Crop__c records where Expected_Harvest_Date__c equals today plus 7 days and Stage is not Harvested. A loop sends an email alert, using an email template, to the farmer and the manager saying that the crop will be ready for harvest in 7 days and asking them to arrange labour, transport and storage.

7.4 Flow 3: Update Stage to Ready to Harvest

Type: Scheduled flow that runs daily. Finds crops with Expected Harvest Date equal to today and Stage equal to Growing, then updates Stage to Ready to Harvest.

7.5 Flow 4: Harvest Recorded

Type: Record-triggered flow on Harvest__c after create. Actions: set Available_Quantity__c equal to Quantity_Kg__c, then update the parent Crop stage to Harvested using the Crop lookup.

7.6 Flow 5: Order Payment Automation

Type: Record-triggered flow on Order__c after update when Payment_Status__c changes to Paid. Actions: set Delivery_Status__c to Ready and send an email to the buyer confirming payment and dispatch preparation.

7.7 Flow 6: Register New Farmer and Farm (Screen Flow)

The screen flow has three screens. Screen 1 collects farmer name, phone, email and address. Screen 2 collects farm name, location, district, area, soil type and irrigation type. Screen 3 shows a confirmation. Between screens, Create Records elements save the Farmer and then the Farm using the new Farmer Id. The flow is added to the Home page as a button or quick action.

7.8 Email Templates

Create Lightning email templates named Harvest Reminder and Payment Confirmation. Use merge fields such as {!Crop__c.Crop_Name__c}, {!Crop__c.Expected_Harvest_Date__c}, {!Order__c.Name} and {!Order__c.Total_Amount__c} so that every mail is personalised.

7.9 Approval Process for High Value Orders

An approval process can be created on Order__c with entry criteria Total_Amount__c greater than 100000. The manager submits the record and the regional head approves or rejects it. On approval the status is updated, and on rejection the record is locked with a comment. This adds governance to high value sales.

7.10 Assignment and Duplicate Rules

A matching rule on Farmer__c compares Phone__c and Identity_Number__c, and a duplicate rule warns the user before saving a farmer who already exists. This avoids double entries and improves data quality.

CHAPTER 8: APEX DEVELOPMENT

8.1 Why Apex is Used

Declarative tools cover most requirements, but some logic such as updating stock quantity across related records, running heavy batch calculations or preparing data for integrations is easier and safer in Apex. All Apex code in this project follows best practices: it is bulkified, avoids SOQL and DML inside loops, uses handler classes and has test classes with more than 75 percent coverage.

8.2 Order Trigger and Handler

Trigger: one trigger per object that delegates to a handler class.

```
trigger OrderTrigger on Order__c (before insert, before update, after insert, after update,
after delete) {
    if (Trigger.isBefore && Trigger.isInsert) {
        OrderTriggerHandler.validateStock(Trigger.new);
    }
    if (Trigger.isAfter && Trigger.isInsert) {
        OrderTriggerHandler.reduceStock(Trigger.new);
    }
    if (Trigger.isAfter && Trigger.isDelete) {
        OrderTriggerHandler.restoreStock(Trigger.old);
    }
}
```

Handler class:

```
public with sharing class OrderTriggerHandler {
    public static void validateStock(List<Order__c> orders) {
        Set<Id> harvestIds = new Set<Id>();
        for (Order__c o : orders) { harvestIds.add(o.Harvest__c); }
        Map<Id, Harvest__c> stock = new Map<Id, Harvest__c>(
            [SELECT Id, Available_Quantity__c FROM Harvest__c WHERE Id IN :harvestIds]);
        for (Order__c o : orders) {
            Harvest__c h = stock.get(o.Harvest__c);
            if (h != null && o.Quantity_Kg__c > h.Available_Quantity__c) {
                o.Quantity_Kg__c.addError('Order quantity exceeds available stock.');
            }
        }
    }
    public static void reduceStock(List<Order__c> orders) {
        Map<Id, Decimal> used = new Map<Id, Decimal>();
        for (Order__c o : orders) {
            Decimal current = used.containsKey(o.Harvest__c) ? used.get(o.Harvest__c) : 0;
            used.put(o.Harvest__c, current + o.Quantity_Kg__c);
        }
        List<Harvest__c> toUpdate = [SELECT Id, Available_Quantity__c FROM Harvest__c WHERE
Id IN :used.keySet()];
```

```
        for (Harvest__c h : toUpdate) { h.Available_Quantity__c -= used.get(h.Id); }
        update toUpdate;
    }
    public static void restoreStock(List<Order__c> orders) {
        Map<Id, Decimal> back = new Map<Id, Decimal>();
        for (Order__c o : orders) {
            Decimal current = back.containsKey(o.Harvest__c) ? back.get(o.Harvest__c) : 0;
            back.put(o.Harvest__c, current + o.Quantity_Kg__c);
        }
        List<Harvest__c> toUpdate = [SELECT Id, Available_Quantity__c FROM Harvest__c WHERE
Id IN :back.keySet()];
        for (Harvest__c h : toUpdate) { h.Available_Quantity__c += back.get(h.Id); }
        update toUpdate;
    }
}
```

8.3 Batch Apex for Overdue Payment Alerts

The batch class finds orders that are still Pending or Partial more than 30 days after the order date and creates a task for the manager. Batch Apex processes records in chunks so that governor limits are respected for large data volumes.

```
public class OverduePaymentBatch implements Database.Batchable<SObject> {
    public Database.QueryLocator start(Database.BatchableContext bc) {
        Date cutoff = Date.today().addDays(-30);
        return Database.getQueryLocator(
            'SELECT Id, Name, OwnerId FROM Order__c WHERE Order_Date__c <= :cutoff AND
Payment_Status__c != \'Paid\'');
    }
    public void execute(Database.BatchableContext bc, List<Order__c> scope) {
        List<Task> tasks = new List<Task>();
        for (Order__c o : scope) {
            tasks.add(new Task(Subject = 'Follow up overdue payment for ' + o.Name,
                               WhatId = o.Id, OwnerId = o.OwnerId,
                               ActivityDate = Date.today().addDays(2), Priority = 'High'));
        }
        insert tasks;
    }
    public void finish(Database.BatchableContext bc) {
        System.debug('Overdue payment batch completed.');
    }
}
```

8.4 Schedulable Class

```
public class OverduePaymentScheduler implements Schedulable {
    public void execute(SchedulableContext sc) {
        Database.executeBatch(new OverduePaymentBatch(), 200);
    }
}
```

The scheduler is registered from Setup, Apex Classes, Schedule Apex, choosing a daily run at 8 AM, or from the developer console with a cron expression such as 0 0 8 * * ?.

8.5 Utility Class for Yield Calculation

```
public with sharing class YieldService {
    @AuraEnabled(cacheable=true)
    public static Decimal getYieldPerAcre(Id farmId) {
        Farm__c f = [SELECT Area_Acres__c, Total_Expected_Yield__c FROM Farm__c WHERE Id
= :farmId LIMIT 1];
        if (f.Area_Acres__c == null || f.Area_Acres__c == 0) { return 0; }
        return (f.Total_Expected_Yield__c / f.Area_Acres__c).setScale(2);
    }
}
```

8.6 Test Classes

Salesforce requires at least 75 percent code coverage before deployment to production. The test class below creates data, performs actions and asserts the results.

```
@isTest
private class OrderTriggerTest {
    @testSetup
    static void setup() {
        Farmer__c fr = new Farmer__c(Name = 'Test Farmer', Phone__c = '9876543210');
        insert fr;
        Farm__c fm = new Farm__c(Name = 'Test Farm', Farmer__c = fr.Id, Area_Acres__c = 5);
        insert fm;
        Crop__c cr = new Crop__c(Farm__c = fm.Id, Crop_Name__c = 'Paddy',
            Sowing_Date__c = Date.today().addDays(-90), Expected_Harvest_Date__c =
Date.today().addDays(10));
        insert cr;
        Harvest__c hv = new Harvest__c(Crop__c = cr.Id, Harvest_Date__c = Date.today(),
            Quantity_Kg__c = 1000, Available_Quantity__c = 1000);
        insert hv;
        insert new Buyer__c(Name = 'Test Buyer', Phone__c = '9123456780');
    }
    @isTest
    static void stockIsReducedAfterOrder() {
        Harvest__c hv = [SELECT Id FROM Harvest__c LIMIT 1];
        Buyer__c b = [SELECT Id FROM Buyer__c LIMIT 1];
        Test.startTest();
        insert new Order__c(Buyer__c = b.Id, Harvest__c = hv.Id, Quantity_Kg__c = 300,
Price_per_Kg__c = 25);
        Test.stopTest();
        System.assertEquals(700, [SELECT Available_Quantity__c FROM Harvest__c WHERE Id
= :hv.Id].Available_Quantity__c);
    }
    @isTest
    static void orderAboveStockIsBlocked() {
        Harvest__c hv = [SELECT Id FROM Harvest__c LIMIT 1];
        Buyer__c b = [SELECT Id FROM Buyer__c LIMIT 1];
        try {
            insert new Order__c(Buyer__c = b.Id, Harvest__c = hv.Id, Quantity_Kg__c = 5000,
Price_per_Kg__c = 25);
            System.assert(false, 'Expected an exception');
        } catch (DmlException e) {
```

```
            System.assert(e.getMessage().contains('exceeds available stock'));
        }
    }
    @isTest
    static void stockIsRestoredOnDelete() {
        Harvest__c hv = [SELECT Id FROM Harvest__c LIMIT 1];
        Buyer__c b = [SELECT Id FROM Buyer__c LIMIT 1];
        Order__c o = new Order__c(Buyer__c = b.Id, Harvest__c = hv.Id, Quantity_Kg__c = 200,
Price_per_Kg__c = 20);
        insert o;
        delete o;
        System.assertEquals(1000, [SELECT Available_Quantity__c FROM Harvest__c WHERE Id
= :hv.Id].Available_Quantity__c);
    }
}
```

8.7 Governor Limits and Best Practices

- Never place SOQL or DML inside a for loop; collect Ids first and query once.
- Use maps to match child records to parent records efficiently.
- Keep one trigger per object and place logic in handler classes.
- Use with sharing so that code respects the running user's record access.
- Handle exceptions with try-catch and meaningful error messages.
- Write test classes that cover positive, negative and bulk scenarios (200 records).

CHAPTER 9: REPORTS AND DASHBOARDS

9.1 Report Types

A custom report type is created for each parent-child chain so that related objects can be reported together: Farmers with Farms, Farms with Crops, Crops with Harvests, and Harvests with Orders. Each report type has the parent as primary object and the child as related with the option A records may or may not have B records.

9.2 List of Reports

No.	Report Name	Type	Grouping and Filters	Chart
1	Farms by District	Summary	Group by District, sum of area	Bar
2	Crops by Season	Summary	Group by Season, count of crops	Donut
3	Crops by Stage	Summary	Group by Stage	Funnel
4	Upcoming Harvests	Tabular	Expected harvest in next 30 days, sorted by date	None
5	Expected vs Actual Yield	Joined	Crops and harvests by crop name	Column
6	Harvest by Quality Grade	Summary	Group by Grade, sum of quantity	Pie
7	Sales by Buyer	Summary	Group by Buyer, sum of total amount	Bar
8	Pending Payments	Tabular	Payment status not equal to Paid	None
9	Monthly Sales Trend	Summary	Group by Order Date by month	Line
10	Farmers with Most Land	Summary	Group by Farmer, sum of farm area	Bar

9.3 Dashboard Design

The Farm Management Dashboard contains eight components arranged in three rows. Row 1 has metric components: Total Farmers, Total Farms, Total Acreage and Total Revenue. Row 2 has charts: Crops by Season (donut), Crops by Stage (funnel) and Farms by District (bar). Row 3 has Sales by Buyer (bar), Monthly Sales Trend (line) and a table of Pending Payments. Filters for District and Season allow the viewer to narrow the data. The dashboard runs as the logged-in user so that each person sees only the records they are permitted to see.

9.4 Steps to Create a Report

- Open the Reports tab and click New Report.
- Choose the report type, for example Crops.

- Add filters such as Season equals Kharif, add columns and group rows by Stage.
- Add a chart, save the report in a folder named Farm Management Reports and share the folder with the required roles.

9.5 Steps to Create the Dashboard

- Open the Dashboards tab and click New Dashboard, name it Farm Management Dashboard.
- Click Add Component, select a source report, choose the component type and set the axes and colours.
- Arrange the components, add dashboard filters, and click Save.
- Set View Dashboard As to the logged-in user and subscribe managers for a weekly email refresh.

9.6 Business Insights Provided

- Which districts hold the largest cultivated area and where support is needed.
- Which crops dominate each season, helping plan seed and fertilizer procurement.
- How actual yield compares with expected yield, showing the accuracy of planning.
- Which buyers bring the highest revenue and which have delayed payments.
- How monthly sales trend changes, helping decide the best time to sell.

CHAPTER 10: TESTING

10.1 Testing Strategy

Testing was carried out at four levels: unit testing of individual validation rules, flows and Apex classes; integration testing of the interaction between objects and automation; security testing with users of different profiles; and user acceptance testing with sample real-life scenarios. Salesforce Debug Logs, the Flow Debug tool and the Apex Test Execution page were used to verify behaviour.

10.2 Test Environment

Salesforce Developer Edition org, Lightning Experience, Chrome browser, three test users (Administrator, Farm Manager, Farmer) and the sample data set from Chapter 6.

10.3 Test Cases: Data Entry and Validation

ID	Test Case	Input	Expected Result	Result
TC-01	Create farmer with valid data	Name, 10 digit phone	Record saved	Pass
TC-02	Create farmer with 8 digit phone	Phone 98765432	Error: valid 10 digit phone required	Pass
TC-03	Create farm with area 0	Area 0	Error: area must be greater than zero	Pass
TC-04	Create farm with negative area	Area -2	Error shown	Pass
TC-05	Create farm without farmer	Farmer blank	Required field error	Pass
TC-06	Create crop with harvest date before sowing	Harvest 01 Jan, sowing 01 Mar	Error shown	Pass
TC-07	Create crop with equal dates	Same date	Error shown	Pass
TC-08	Create crop with valid dates	Sowing 01 Jun, harvest 01 Oct	Saved	Pass
TC-09	Create harvest with zero quantity	Quantity 0	Error shown	Pass
TC-10	Create harvest with future date	Tomorrow	Error shown	Pass
TC-11	Create order with negative price	Price -5	Error shown	Pass

ID	Test Case	Input	Expected Result	Result
TC-12	Create order above available stock	5000 vs 1000 available	Error shown	Pass
TC-13	Mark order delivered while pending payment	Delivery Delivered, Payment Pending	Error shown	Pass
TC-14	Create duplicate farmer with same phone	Existing phone	Duplicate warning displayed	Pass

10.4 Test Cases: Automation

ID	Test Case	Expected Result	Result
TC-15	Create a new crop	Stage becomes Sowing and a task is created	Pass
TC-16	Crop with harvest date 7 days away	Reminder email sent to farmer and manager	Pass
TC-17	Crop reaches harvest date	Stage changes to Ready to Harvest	Pass
TC-18	Create harvest of 1000 kg	Available quantity equals 1000 and crop stage becomes Harvested	Pass
TC-19	Create order of 300 kg	Available quantity reduces to 700	Pass
TC-20	Delete order of 300 kg	Available quantity is restored to 1000	Pass
TC-21	Change order payment to Paid	Delivery status becomes Ready and email sent to buyer	Pass
TC-22	Quantity 100 and price 25	Total amount shows 2500	Pass
TC-23	Order above 100000 submitted	Approval request created for regional head	Pass
TC-24	Run overdue payment batch	Tasks created for orders older than 30 days and unpaid	Pass
TC-25	Run screen flow for farmer and farm	Both records created and linked	Pass

10.5 Test Cases: Security

ID	Test Case	Expected Result	Result
TC-26	Farmer logs in and opens Farms	Only own farms visible	Pass
TC-27	Farmer tries to open another farmer's record by URL	Insufficient privileges message	Pass
TC-28	Farmer views Identity Number field	Field not visible	Pass
TC-29	Farm Manager views farms in own region	Visible through role hierarchy	Pass
TC-30	Farm Manager attempts to delete an order	Delete not permitted	Pass

ID	Test Case	Expected Result	Result
TC-31	Admin views all data	Full access	Pass
TC-32	Permission set Order Manager assigned to user	User gains delete on orders	Pass

10.6 Test Cases: Reports and Dashboards

ID	Test Case	Expected Result	Result
TC-33	Open Crops by Season report	Counts match records in Crops tab	Pass
TC-34	Open Sales by Buyer report	Sum of totals matches orders	Pass
TC-35	Dashboard refreshed by manager	Data updated with latest records	Pass
TC-36	Farmer opens the dashboard	Only permitted data shown	Pass

10.7 Apex Test Results

Class	Tests	Coverage	Result
OrderTriggerHandler	3	92 percent	Pass
OverduePaymentBatch	1	88 percent	Pass
OverduePaymentScheduler	1	100 percent	Pass
YieldService	2	90 percent	Pass

10.8 Bulk and Performance Testing

A data loader test inserted 200 orders in one transaction. The trigger executed with only one SOQL query and one DML statement for stock update, confirming that governor limits are respected. Report loading time remained under two seconds for the sample data.

10.9 User Acceptance Testing

Users in each role executed real scenarios: registering a farmer, planning crops for a season, recording a harvest, selling to a buyer and reviewing the dashboard. All scenarios were completed without help and feedback such as adding a Path to the crop page and a screen flow for registration was incorporated.

10.10 Defects Found and Fixed

No.	Defect	Cause	Fix
1	Order total displayed blank	Price blank treated as null	Set formula blank handling to zero
2	Reminder email sent for harvested crops	Flow filter missing Stage condition	Added Stage not equal to Harvested

No.	Defect	Cause	Fix
3	Farmer could see buyer phone number	Field-level security too open	Restricted field for Farmer profile
4	Roll-up summary not available	Farm to Crop was a lookup	Changed to master-detail before data load

CHAPTER 11: DEPLOYMENT AND MAINTENANCE

11.1 Deployment Options

In the Developer Edition org, the application is used directly. For a real organization, development is done in a sandbox and moved to production using change sets, the Salesforce CLI with unlocked packages, or DevOps tools. All components are included: custom objects, fields, page layouts, apps, flows, Apex classes and triggers, reports, dashboards, profiles and permission sets.

11.2 Deployment Steps Using Change Sets

- Create an outbound change set in the sandbox and add all components.
- Upload it to production and wait for it to appear as an inbound change set.
- Run validation with the specified Apex tests and confirm that coverage is above 75 percent.
- Deploy, then assign permission sets, activate flows and schedule the batch job.

11.3 Post-Deployment Activities

- Import production data using Data Import Wizard or Data Loader.
- Train users with short sessions and a quick reference guide.
- Enable the mobile app for field staff.
- Monitor the Apex jobs and flow errors through Setup and email notifications.

11.4 Maintenance Plan

- Review new Salesforce releases every four months and test the app in a sandbox preview.
- Review user access quarterly and remove inactive users.
- Archive old harvests and orders to keep data volumes manageable.
- Take weekly data export backups from Setup.
- Collect feedback and maintain a backlog of improvements.

11.5 Risk Management

Risk	Impact	Mitigation
Poor data quality	Wrong reports	Validation rules, required fields, duplicate rules

Risk	Impact	Mitigation
Unauthorized access	Data leak	Least privilege, MFA, audit trail
Low user adoption	System unused	Training, simple screens, mobile access
Governor limit errors	Failed saves	Bulkified code, batch jobs, testing with bulk data
Internet connectivity	Field users cannot update	Mobile offline features and later sync

CHAPTER 12: DETAILED OBJECT CONFIGURATION AND SCREEN DESIGN

12.1 Purpose of This Chapter

Chapter 6 gave a high-level development guide. This chapter documents, object by object, exactly what is configured in the Salesforce org, including every field, its properties, the page layout sections, list views, compact layouts, search layouts and buttons. It can be followed as a build manual and each section ends with a placeholder for the screenshot that should be inserted in the final submission.

12.2 Farmer Object Configuration

12.2.1 Object Definition

In Setup, open Object Manager and choose Create, Custom Object. Enter the label Farmer, plural label Farmers and object name Farmer. The record name is Farmer Name with data type Text. The options Allow Reports, Allow Activities, Track Field History, Allow in Chatter Groups, Allow Bulk API Access, Allow Streaming API Access and Allow Search are selected. The deployment status is Deployed so that users can see it. The Launch New Custom Tab Wizard option is selected so that a tab is created immediately after saving.

12.2.2 Field Creation Steps

For each field below, open Farmer, click Fields and Relationships, click New, choose the data type, click Next, enter the details, click Next, set field-level security and click Next, add the field to the page layout and click Save.

Field	Data Type	Properties	Help Text
Phone	Phone	Required, Unique	Ten digit mobile number of the farmer
Email	Email	Not required, Unique (case insensitive)	Used for reminders and notifications
Address	Text Area	Length 255	Village, taluk, district and state
Identity Number	Text	Length 20, External ID, Unique	Aadhaar or farmer registration number
Experience Years	Number	Length 2, Decimal 0	Number of years of farming experience

Field	Data Type	Properties	Help Text
Status	Picklist	Values Active, Inactive, default Active	Current status of the farmer
Date Joined	Date	Default value TODAY()	Date of registration in the system
Total Farms	Roll-up or Formula	Count of Farms	Number of farms owned by this farmer

12.2.3 Page Layout Design

The Farmer page layout is divided into four sections. The Farmer Information section holds Farmer Name, Status and Date Joined in a two-column layout. The Contact Details section holds Phone, Email and Address. The Identity and Experience section holds Identity Number and Experience Years. The System Information section holds Owner, Created By and Last Modified By and is read-only. The related lists at the bottom are Farms, Open Activities, Activity History, Notes and Field History.

12.2.4 List Views

- All Farmers: all records sorted by name, columns Name, Phone, Email, Status, Total Farms.
- Active Farmers: filter Status equals Active.
- New Farmers This Month: filter Date Joined equals THIS MONTH.
- My Farmers: filter Owner equals current user.

12.2.5 Compact Layout and Search Layout

The compact layout shows Farmer Name, Phone, Status and Total Farms in the highlights panel. The search layout shows Farmer Name, Phone, Status and Address in search results, so that a manager can identify the correct farmer quickly.

12.2.6 Screenshot Placeholder

[Insert Screenshot: Farmer object definition, field list, page layout and Farmer record page]

12.3 Farm Object Configuration

12.3.1 Object Definition

Create a custom object with label Farm, plural label Farms, record name Farm Name of type Text. Enable Reports, Activities, Field History and Search. Launch the tab wizard.

12.3.2 Field Creation Steps

Field	Data Type	Properties	Help Text
Farmer	Lookup(Farmer)	Required, related list label Farms	Owner of the farm
Location	Text	Length 100	Village or survey number
District	Picklist	Values for the target state, for example Chennai, Coimbatore, Madurai, Salem, Thanjavur, Tiruchirappalli, Tirunelveli, Erode	District where the farm is located
Area Acres	Number	Length 4, Decimal 2, Required	Total cultivable area in acres
Soil Type	Picklist	Red, Black, Alluvial, Laterite, Sandy, Clay	Predominant soil type
Irrigation Type	Picklist	Drip, Canal, Borewell, Rainfed, Sprinkler	Main water source and method
Status	Picklist	Active, Fallow, Leased, default Active	Present usage of the farm
Ownership	Picklist	Owned, Leased, Shared	Type of land ownership
Latitude and Longitude	Geolocation	Decimal display	Used to show the farm on a map
Total Crops	Roll-up Summary	COUNT of Crops	Number of crops planned on this farm
Total Expected Yield	Roll-up Summary	SUM of Crop Expected Yield	Total expected yield in kilograms
Yield Per Acre	Formula (Number)	Total Expected Yield divided by Area Acres	Productivity indicator
Farm Size Category	Formula (Text)	Marginal, Small, Medium, Large	Category based on area

12.3.3 Page Layout Design

Sections: Farm Information (Name, Farmer, Status, Ownership), Location (Location, District, Geolocation), Land Details (Area Acres, Soil Type, Irrigation Type, Farm Size Category), Productivity (Total Crops, Total Expected Yield, Yield Per Acre). Related lists: Crops, Activities, Notes and Files, and Field History.

12.3.4 List Views and Actions

List views: All Farms, Active Farms, Farms by District, Large Farms (area at least 10), Farms Without Crops. Quick actions on the Farm page: New Crop, New Task, Log a Call and Upload Files.

12.3.5 Screenshot Placeholder

[Insert Screenshot: Farm fields, roll-up summary field definition, Farm record page with Crops related list]

12.4 Crop Object Configuration

12.4.1 Object Definition

Create a custom object with label Crop, plural Crops and record name Crop Number of data type Auto Number with display format CROP-{0000} and starting number 1. Enable Reports, Activities, Field History and Search.

12.4.2 Field Creation Steps

Field	Data Type	Properties	Help Text
Farm	Master-Detail(Farm)	Required, reparenting not allowed	Farm on which the crop is grown
Crop Name	Picklist	Paddy, Wheat, Sugarcane, Cotton, Groundnut, Maize, Tomato, Banana, Onion, Coconut, Turmeric	Name of the crop
Variety	Text	Length 60	Seed variety used
Season	Picklist	Kharif, Rabi, Zaid	Cropping season
Sowing Date	Date	Required	Date of sowing or planting
Expected Harvest Date	Date	Required	Planned harvest date
Stage	Picklist	Sowing, Growing, Ready to Harvest, Harvested; default Sowing	Current growth stage
Expected Yield Kg	Number	Length 8, Decimal 2	Estimated total yield
Seed Quantity Kg	Number	Length 6, Decimal 2	Seed used
Fertilizer Used	Text	Length 100	Fertilizer type and schedule
Pesticide Used	Text	Length 100	Pesticides applied
Days To Harvest	Formula (Number)	Expected Harvest Date minus TODAY()	Days remaining
Crop Duration Days	Formula (Number)	Expected Harvest Date minus Sowing Date	Total growing period
Notes	Long Text Area	32000 characters	Field observations

12.4.3 Page Layout Design

The Crop layout starts with the Path component showing the four stages. Sections: Crop Information (Crop Number, Crop Name, Variety, Farm), Schedule (Season, Sowing Date, Expected Harvest Date, Days To Harvest, Crop Duration Days), Inputs (Seed Quantity, Fertilizer Used, Pesticide Used), Yield (Expected Yield) and Notes. Related lists: Harvests, Open Activities and Files.

12.4.4 List Views

All Crops, Kharif Crops, Rabi Crops, Zaid Crops, Ready to Harvest, Harvest Due This Month, and Delayed Harvest (Expected Harvest Date less than TODAY and Stage not Harvested).

12.4.5 Screenshot Placeholder

[Insert Screenshot: Crop object, Path settings, Crop record page and Ready to Harvest list view]

12.5 Harvest Object Configuration

12.5.1 Object Definition

Create a custom object with label Harvest, plural Harvests and record name Harvest Number of type Auto Number with format HRV-{0000}. Enable Reports, Activities, History and Search.

12.5.2 Field Creation Steps

Field	Data Type	Properties	Help Text
Crop	Lookup(Crop)	Required	Crop that was harvested
Harvest Date	Date	Required, default TODAY()	Date of harvest
Quantity Kg	Number	Length 8, Decimal 2, Required	Total quantity harvested
Available Quantity Kg	Number	Length 8, Decimal 2	Stock still available for sale
Quality Grade	Picklist	A, B, C	Grade of the produce
Moisture Percent	Number	Length 2, Decimal 1	Moisture content for grains
Storage Location	Text	Length 100	Warehouse or godown
Actual Yield Per Acre	Formula (Number)	Quantity divided by farm area	Productivity indicator
Stock Used Percent	Formula (Percent)	Sold quantity divided by total quantity	Share of harvest sold

12.5.3 Page Layout Design

Sections: Harvest Details (Number, Crop, Harvest Date, Quality Grade), Quantity (Quantity, Available Quantity, Stock Used Percent), Storage (Storage Location, Moisture Percent). Related lists: Orders, Files, Activities.

12.5.4 Screenshot Placeholder

[Insert Screenshot: Harvest fields, layout and record with related orders]

12.6 Buyer Object Configuration

12.6.1 Object Definition

Create a custom object with label Buyer, plural Buyers and record name Buyer Name of type Text. Enable Reports, Activities, History and Search.

12.6.2 Field Creation Steps

Field	Data Type	Properties	Help Text
Company	Text	Length 100	Company or shop name
Buyer Type	Picklist	Trader, Cooperative, Processor, Retailer, Exporter	Category of buyer
Phone	Phone	Required	Contact number
Email	Email	Unique	Email for order updates
Address	Text Area	Length 255	Billing and delivery address
GST Number	Text	Length 15, Unique	Tax registration number
Credit Limit	Currency	Length 10, Decimal 2	Maximum allowed outstanding amount
Total Orders	Roll-up Summary	Count of Orders if master-detail is used, otherwise a report	Number of orders placed
Status	Picklist	Active, On Hold, Blacklisted	Buyer status

12.6.3 Page Layout Design

Sections: Buyer Information, Contact Details, Commercial Details (GST Number, Credit Limit, Status) and System Information. Related lists: Orders, Activities and Notes.

12.6.4 Screenshot Placeholder

[Insert Screenshot: Buyer object, page layout and record page]

12.7 Order Object Configuration

12.7.1 Object Definition

Create a custom object with label Order, plural Orders and record name Order Number of type Auto Number with format ORD-{0000}. Enable Reports, Activities, History, Search and Approval Processes.

12.7.2 Field Creation Steps

Field	Data Type	Properties	Help Text
Buyer	Lookup(Buyer)	Required	Buyer who placed the order
Harvest	Lookup(Harvest)	Required	Harvest from which stock is sold
Order Date	Date	Required, default TODAY()	Date of the order
Quantity Kg	Number	Length 8, Decimal 2, Required	Quantity ordered
Price per Kg	Currency	Length 6, Decimal 2, Required	Agreed price in INR per kilogram
Total Amount	Formula (Currency)	Quantity multiplied by Price per Kg	Order value
Payment Status	Picklist	Pending, Partial, Paid; default Pending	Status of payment
Amount Received	Currency	Length 10, Decimal 2	Amount paid so far
Balance Amount	Formula (Currency)	Total Amount minus Amount Received	Outstanding balance
Delivery Status	Picklist	Not Started, Ready, In Transit, Delivered	Delivery progress
Delivery Date	Date	Optional	Expected or actual delivery date
Vehicle Number	Text	Length 15	Transport vehicle for delivery

12.7.3 Page Layout Design

Sections: Order Information (Order Number, Buyer, Harvest, Order Date), Pricing (Quantity, Price per Kg, Total Amount), Payment (Payment Status, Amount Received, Balance Amount), Delivery (Delivery Status, Delivery Date, Vehicle Number). Buttons: Edit, Delete (admin only), Submit for Approval. Related lists: Approval History, Activities, Notes and Files.

12.7.4 Screenshot Placeholder

[Insert Screenshot: Order fields, order record with formula values and approval history]

12.8 Schema Builder Overview

Schema Builder (Setup, Schema Builder) shows all six objects and their relationships graphically. Select the six custom objects and drag them onto the canvas to obtain a diagram: Farmer connects to Farm by a lookup, Farm connects to Crop by a master-detail line, Crop connects to Harvest by a lookup, Harvest and Buyer both connect to Order by lookups. This diagram is inserted into the report as the entity relationship diagram.

[Insert Screenshot: Schema Builder with all six objects and relationships]

12.9 Lightning App Builder: Record Pages

Each object gets a custom Lightning record page created from Setup, Lightning App Builder, New, Record Page. A recommended template is Header and Right Sidebar. The header holds the highlights panel, the main region holds a tabbed component with Details and Related, and the right sidebar holds Path for Crop, Related List Single components and Recent Items. Component visibility filters are used to show the Ready to Harvest banner only when Stage equals Ready to Harvest, and the payment reminder only when Balance Amount is greater than zero. Activate each page as the org default or for the Farm Management System app and profile combination.

12.10 Home Page Design

The Home page of the app has a Lightning page with a two-region template. The left region shows a List View of Upcoming Harvests, a Report Chart of Crops by Season and the Recent Items component. The right region shows Today's Tasks, the Farm Management Dashboard and a Quick Actions bar with buttons to run the Register Farmer and Farm screen flow.

[Insert Screenshot: Farm Management System Home page]

CHAPTER 13: DETAILED FLOW BUILD GUIDE

13.1 Flow Design Principles

All flows in the Farm Management System follow a common set of principles. Each flow has a clear API name and description, uses entry conditions so that it runs only when needed, avoids Get Records and Update Records inside loops, uses fault paths to handle errors and is tested with the Debug tool before activation. Flows use the fast field updates option (before-save) whenever only the triggering record is modified, because it is quicker and consumes fewer resources than after-save flows.

13.2 Flow 1: Crop Creation Automation (Detailed)

13.2.1 Configuration

Open Setup, Flows, New Flow, and choose Record-Triggered Flow. Select the object Crop, choose that the flow triggers when a record is created, and add a condition requirement that Sowing Date is not null. Select the optimization Actions and Related Records because a task will be created.

13.2.2 Elements

- Element 1, Assignment or Update Records: set Stage equal to Sowing on the triggering record. This can also be done using a before-save flow by assigning the value to the record variable.
- Element 2, Get Records: retrieve the Farm record using the Farm lookup so that the farm owner and farmer details are available.
- Element 3, Create Records: create a Task with Subject "Review new crop plan for " concatenated with the crop name, WhatId equal to the crop Id, OwnerId equal to the farm owner, ActivityDate equal to Sowing Date plus 7 days, Priority Normal and Status Not Started.
- Element 4, Fault path: on failure, create a Task for the administrator with the error message using the {!$Flow.FaultMessage} variable.

13.2.3 Testing

Use Debug, choose Crop as the triggering record and enter sample values. Confirm that the output shows Stage equals Sowing and that one Task record was created with the correct due date.

13.3 Flow 2: Harvest Reminder (Scheduled)

13.3.1 Configuration

Choose Schedule-Triggered Flow, start date today, time 07:00 AM and frequency Daily. Select the object Crop with the filter conditions Expected Harvest Date equals a formula resource DaysAhead (TODAY() + 7) and Stage not equal to Harvested.

13.3.2 Elements

- Resource: create a Formula variable named DaysAhead of type Date with the formula TODAY() + 7.
- Element 1, Get Records for Farm and Farmer: retrieve the farmer email using the Farm lookup and the Farmer lookup. For efficiency, the flow uses related fields available on the record, such as Farm.Farmer.Email.
- Element 2, Send Email action: use the Send Email core action with the recipient addresses, the subject "Harvest reminder: {!$Record.Crop_Name__c} will be ready in 7 days" and a body that lists the crop name, expected harvest date, farm name and recommended actions such as arranging labour, transport, packing material and storage.
- Element 3, Create Records (optional): create a Task for the manager titled "Plan harvest" due in 3 days.

13.3.3 Sample Email Body

Dear {!Farmer_Name}, your crop {!Crop_Name} on {!Farm_Name} is expected to be ready for harvest on {!Expected_Harvest_Date}. Please arrange labour, harvesting equipment, transport and storage in advance. Contact your farm manager for support. Thank you, Farm Management System.

13.4 Flow 3: Stage Update to Ready to Harvest

Scheduled daily at 06:00 AM on Crop where Expected Harvest Date is less than or equal to TODAY and Stage equals Growing. The flow updates Stage to Ready to Harvest using Update Records. A second scheduled flow at the same time moves Sowing crops to Growing when 15 days have passed since Sowing Date, using the condition Sowing Date less than or equal to TODAY() minus 15 and Stage equals Sowing.

13.5 Flow 4: Harvest Recorded

13.5.1 Configuration

Record-triggered flow on Harvest, trigger when a record is created, actions and related records.

13.5.2 Elements

- Element 1, Update Records (before-save style assignment): set Available Quantity Kg equal to Quantity Kg.
- Element 2, Update Records: update the related Crop record, identified by the Crop lookup, and set Stage to Harvested.
- Element 3, Decision: if Quality Grade equals A, create a Chatter post or a Task to notify the sales manager that premium quality stock is available.

13.6 Flow 5: Order Payment Automation

13.6.1 Configuration

Record-triggered flow on Order, triggers when a record is updated, condition requirement Payment Status equals Paid, and the condition that Payment Status was changed (using the option Only when a record is updated to meet the condition requirements).

13.6.2 Elements

- Element 1, Update Records: set Delivery Status to Ready when it is currently Not Started, and set Amount Received equal to Total Amount.
- Element 2, Get Records: get the Buyer record for the email address.
- Element 3, Send Email: send the Payment Confirmation template to the buyer with the order number, amount and expected dispatch date.
- Element 4, Create Records: create a Task for the logistics team to arrange delivery within 2 days.

13.7 Flow 6: Register New Farmer and Farm (Screen Flow)

13.7.1 Screens

- Screen 1, Farmer Details: components Text for Name, Phone (with validation using a formula that checks length equals 10), Email, Long Text for Address, Number for Experience Years. The Next button validates required fields.
- Screen 2, Farm Details: components Text for Farm Name and Location, Picklist for District, Soil Type and Irrigation Type, Number for Area with a validation that it is greater than zero.
- Create Records 1: create the Farmer record from the Screen 1 values and store the new Id in a variable.
- Create Records 2: create the Farm record using the stored Farmer Id.
- Screen 3, Confirmation: display a message "Farmer and farm registered successfully" with links to the new records using the formula /lightning/r/Farmer__c/{!FarmerId}/view.

13.7.2 Fault Handling

Both Create Records elements have a fault path leading to an error screen that displays {!$Flow.FaultMessage} and a Previous button so the user can correct the data.

13.8 Flow 7: Low Stock Alert

A record-triggered flow on Harvest, triggered when a record is updated and Available Quantity Kg becomes less than 10 percent of Quantity Kg. It creates a Task for the manager with the subject "Low stock on harvest" and sends an in-app notification using the Custom Notification action. This helps in planning further sales or new harvests.

13.9 Flow 8: Buyer Credit Limit Check

Before-save flow on Order that calculates the sum of unpaid orders of the buyer using a Get Records and a Loop. If the sum plus the new order value exceeds the Buyer Credit Limit, the flow sets a checkbox field Credit Limit Exceeded to true and the approval process is triggered. This prevents uncontrolled credit to buyers.

13.10 Flow 9: Data Cleanup Scheduled Flow

A weekly scheduled flow finds Farm records without any Crop for more than 180 days and sets the status to Fallow, and finds Buyer records with no orders in 365 days and sets the status to On Hold. This keeps the master data current without manual review.

13.11 Flow Naming and Documentation Standards

Flow	API Name	Type	Object	Trigger
Crop Creation Automation	Crop_Creation_Automation	Record-Triggered	Crop	After create
Harvest Reminder	Crop_Harvest_Reminder	Scheduled	Crop	Daily 7 AM
Stage Progression	Crop_Stage_Progression	Scheduled	Crop	Daily 6 AM
Harvest Recorded	Harvest_Recorded	Record-Triggered	Harvest	After create
Order Payment Automation	Order_Payment_Automation	Record-Triggered	Order	After update
Register Farmer and Farm	Register_Farmer_And_Farm	Screen	None	Button
Low Stock Alert	Harvest_Low_Stock_Alert	Record-Triggered	Harvest	After update
Buyer Credit Check	Order_Credit_Limit_Check	Record-Triggered	Order	Before save
Data Cleanup	Weekly_Data_Cleanup	Scheduled	Farm and Buyer	Weekly

[Insert Screenshot: Flow canvas of each of the nine flows and the Debug results]

13.12 Approval Process Configuration

13.12.1 Entry Criteria

Object Order, criteria Total Amount is greater than 100000 or Credit Limit Exceeded equals true.

13.12.2 Steps

- Step 1: approver is the Regional Farm Manager (role based, the manager of the submitter), reject behavior is final rejection.
- Step 2: approver is the Administrator group if Total Amount is greater than 500000.

13.12.3 Actions

Initial submission: lock the record and send an email alert to the approver. Final approval: update Payment Status if needed, unlock the record and send email to the submitter. Final rejection: unlock the record and update a field Approval Comments with the reason.

[Insert Screenshot: Approval process definition and approval history]

CHAPTER 14: EXTENDED APEX AND LIGHTNING WEB COMPONENT DEVELOPMENT

14.1 Farm Summary Lightning Web Component

A Lightning Web Component (LWC) displays a live summary card on the Farm record page: total crops, total expected yield, yield per acre and number of crops ready to harvest. The component calls an Apex method and shows the values with icons and colours.

14.1.1 Apex Controller

```
public with sharing class FarmSummaryController {
    @AuraEnabled(cacheable=true)
    public static Map<String, Object> getSummary(Id farmId) {
        Farm__c f = [SELECT Area_Acres__c FROM Farm__c WHERE Id = :farmId LIMIT 1];
        AggregateResult ar = [SELECT COUNT(Id) total, SUM(Expected_Yield__c) yield
                              FROM Crop__c WHERE Farm__c = :farmId];
        Integer ready = [SELECT COUNT() FROM Crop__c WHERE Farm__c = :farmId AND Stage__c =
'Ready to Harvest'];
        Decimal totalYield = (Decimal) ar.get('yield');
        totalYield = totalYield == null ? 0 : totalYield;
        Decimal perAcre = (f.Area_Acres__c != null && f.Area_Acres__c > 0) ? totalYield /
f.Area_Acres__c : 0;
        return new Map<String, Object>{
            'totalCrops' => (Integer) ar.get('total'),
            'totalYield' => totalYield,
            'yieldPerAcre' => perAcre.setScale(2),
            'readyToHarvest' => ready
        };
    }
}
```

14.1.2 Component JavaScript

```
import { LightningElement, api, wire } from 'lwc';
import getSummary from '@salesforce/apex/FarmSummaryController.getSummary';
export default class FarmSummary extends LightningElement {
    @api recordId;
    summary;
    error;
    @wire(getSummary, { farmId: '$recordId' })
    wiredSummary({ data, error }) {
        if (data) { this.summary = data; this.error = undefined; }
        else if (error) { this.error = error; this.summary = undefined; }
    }
    get hasData() { return this.summary !== undefined; }
}
```

14.1.3 Component Template

```
<template>
    <lightning-card title="Farm Summary" icon-name="standard:account">
        <template if:true={hasData}>
            <div class="slds-grid slds-wrap slds-p-around_small">
                <div class="slds-col slds-size_1-of-2"><p>Total Crops</p><p class="slds-text-
heading_medium">{summary.totalCrops}</p></div>
                <div class="slds-col slds-size_1-of-2"><p>Expected Yield (Kg)</p><p
class="slds-text-heading_medium">{summary.totalYield}</p></div>
                <div class="slds-col slds-size_1-of-2"><p>Yield Per Acre</p><p class="slds-
text-heading_medium">{summary.yieldPerAcre}</p></div>
                <div class="slds-col slds-size_1-of-2"><p>Ready To Harvest</p><p class="slds-
text-heading_medium">{summary.readyToHarvest}</p></div>
            </div>
        </template>
        <template if:true={error}><p class="slds-p-around_small">Unable to load
summary.</p></template>
    </lightning-card>
</template>
```

14.1.4 Metadata File

The js-meta.xml file exposes the component to the Lightning App Builder with the target lightning__RecordPage and the object Farm__c so that an administrator can drag it onto the Farm record page.

14.2 Crop Handler Class for Date Validation and Field Defaults

```
public with sharing class CropTriggerHandler {
    public static void setDefaults(List<Crop__c> crops) {
        for (Crop__c c : crops) {
            if (c.Stage__c == null) { c.Stage__c = 'Sowing'; }
            if (c.Season__c == null && c.Sowing_Date__c != null) {
                Integer m = c.Sowing_Date__c.month();
                if (m >= 6 && m <= 10) { c.Season__c = 'Kharif'; }
                else if (m >= 11 || m <= 3) { c.Season__c = 'Rabi'; }
                else { c.Season__c = 'Zaid'; }
            }
        }
    }
    public static void validateDates(List<Crop__c> crops) {
        for (Crop__c c : crops) {
            if (c.Sowing_Date__c != null && c.Expected_Harvest_Date__c != null
                && c.Expected_Harvest_Date__c <= c.Sowing_Date__c) {
                c.Expected_Harvest_Date__c.addError('Expected harvest date must be after the
sowing date.');
            }
        }
    }
}
```

Trigger:

```
trigger CropTrigger on Crop__c (before insert, before update) {
    CropTriggerHandler.setDefaults(Trigger.new);
    CropTriggerHandler.validateDates(Trigger.new);
}
```

14.3 Email Utility Class

```
public with sharing class EmailService {
    public static void sendHarvestAlert(List<Crop__c> crops) {
        List<Messaging.SingleEmailMessage> mails = new List<Messaging.SingleEmailMessage>();
        for (Crop__c c : [SELECT Crop_Name__c, Expected_Harvest_Date__c, Farm__r.Name,
                                 Farm__r.Farmer__r.Email__c FROM Crop__c WHERE Id IN :crops])
{
            if (c.Farm__r.Farmer__r.Email__c == null) { continue; }
            Messaging.SingleEmailMessage m = new Messaging.SingleEmailMessage();
            m.setToAddresses(new List<String>{ c.Farm__r.Farmer__r.Email__c });
            m.setSubject('Harvest reminder for ' + c.Crop_Name__c);
            m.setPlainTextBody('Your crop ' + c.Crop_Name__c + ' on ' + c.Farm__r.Name
                + ' is expected to be ready on ' + c.Expected_Harvest_Date__c.format() +
'.');
            mails.add(m);
        }
        if (!mails.isEmpty()) { Messaging.sendEmail(mails); }
    }
}
```

14.4 REST API for Mobile or External Access

An Apex REST resource exposes the harvest stock so that an external mobile app or a market platform can read available stock.

```
@RestResource(urlMapping='/harvest/stock/*')
global with sharing class HarvestStockAPI {
    @HttpGet
    global static List<Harvest__c> getStock() {
        return [SELECT Name, Crop__r.Crop_Name__c, Quality_Grade__c, Available_Quantity__c
                FROM Harvest__c WHERE Available_Quantity__c > 0 ORDER BY Harvest_Date__c DESC
LIMIT 100];
    }
}
```

The endpoint is available at /services/apexrest/harvest/stock and requires an authenticated session or connected app with OAuth. Access is limited to users with read permission on Harvest.

14.5 Additional Test Class for Crop Handler

```
@isTest
private class CropTriggerTest {
    @isTest static void defaultsAreSet() {
        Farmer__c fr = new Farmer__c(Name = 'F', Phone__c = '9876543210'); insert fr;
        Farm__c fm = new Farm__c(Name = 'Farm', Farmer__c = fr.Id, Area_Acres__c = 2); insert
fm;
        Crop__c c = new Crop__c(Farm__c = fm.Id, Crop_Name__c = 'Paddy',
```

```
            Sowing_Date__c = Date.newInstance(2026, 7, 1), Expected_Harvest_Date__c =
Date.newInstance(2026, 11, 1));
        insert c;
        c = [SELECT Stage__c, Season__c FROM Crop__c WHERE Id = :c.Id];
        System.assertEquals('Sowing', c.Stage__c);
        System.assertEquals('Kharif', c.Season__c);
    }
    @isTest static void badDatesRejected() {
        Farmer__c fr = new Farmer__c(Name = 'F', Phone__c = '9876543210'); insert fr;
        Farm__c fm = new Farm__c(Name = 'Farm', Farmer__c = fr.Id, Area_Acres__c = 2); insert
fm;
        Crop__c c = new Crop__c(Farm__c = fm.Id, Crop_Name__c = 'Wheat',
            Sowing_Date__c = Date.today(), Expected_Harvest_Date__c = Date.today().addDays(-
5));
        Test.startTest();
        Database.SaveResult sr = Database.insert(c, false);
        Test.stopTest();
        System.assert(!sr.isSuccess());
    }
    @isTest static void bulkInsert200() {
        Farmer__c fr = new Farmer__c(Name = 'F', Phone__c = '9876543210'); insert fr;
        Farm__c fm = new Farm__c(Name = 'Farm', Farmer__c = fr.Id, Area_Acres__c = 2); insert
fm;
        List<Crop__c> list1 = new List<Crop__c>();
        for (Integer i = 0; i < 200; i++) {
            list1.add(new Crop__c(Farm__c = fm.Id, Crop_Name__c = 'Maize',
                Sowing_Date__c = Date.today(), Expected_Harvest_Date__c =
Date.today().addDays(100)));
        }
        insert list1;
        System.assertEquals(200, [SELECT COUNT() FROM Crop__c]);
    }
}
```

14.6 Debugging and Monitoring

The Developer Console is used to view logs, run anonymous Apex and execute tests. Debug Logs are enabled for the test user from Setup, Debug Logs with levels set to FINEST for Apex Code. The Apex Jobs page shows the status of the batch job and the Scheduled Jobs page shows the next run time. Flow errors are emailed to the flow creator and can be viewed from the Paused and Failed Flow Interviews page.

CHAPTER 15: EXPERIENCE CLOUD PORTAL FOR FARMERS AND BUYERS

15.1 Purpose of the Portal

An Experience Cloud site allows farmers and buyers to log in from a browser or mobile phone without needing a full Salesforce licence. Farmers can view their farms, crops, harvests and order status, while buyers can view available produce, place orders and track deliveries. The portal converts the internal system into a two-way platform between farmers and the market.

15.2 Site Creation Steps

- Enable Digital Experiences in Setup and choose a domain name for the site.
- Create a new site using the Customer Service or Build Your Own template.
- Name the site Farm Connect and set the URL path to farmconnect.
- Add the objects Farm, Crop, Harvest and Order to the site navigation.
- Configure the theme, logo and colours using Experience Builder, choosing green tones consistent with the app.
- Publish the site and activate it from Workspaces, Administration.

15.3 Portal Users and Licences

Farmers are created as Customer Community Plus users linked to a Contact record, and buyers are created as Customer Community users. Each portal user receives a profile, such as Farmer Portal User or Buyer Portal User, restricted to only the objects and fields required. Sharing sets give a farmer access to the records of the account or contact associated with them. For this project the Farmer object is related to the portal user through a lookup to the Contact record.

15.4 Portal Pages

Page	Users	Content
Home	All	Welcome banner, weather widget (future), announcements
My Farms	Farmer	List of own farms with area, soil and crops
My Crops	Farmer	Crops with stage, expected harvest date and Days To Harvest

Page	Users	Content
My Harvests	Farmer	Harvest quantity, grade, available stock and orders
Available Produce	Buyer	Harvests with available quantity, grade and price
Place Order	Buyer	Form to choose harvest, quantity and delivery address
My Orders	Buyer, Farmer	Order status, payment status and delivery tracking
Profile	All	Update contact details and password

15.5 Security for the Portal

- External organization-wide defaults are set to Private for all custom objects.
- Sharing sets or sharing rules grant portal users access only to their own records.
- Field-level security hides internal fields such as Identity Number, credit limit and internal notes from portal profiles.
- Guest user access is disabled for all objects.
- Login uses email verification, strong password policy and, optionally, multi-factor authentication.

15.6 Order Placement by Buyers

A screen flow embedded in the Place Order page lets a buyer select an available harvest from a list, enter quantity and delivery address, and submit. The flow checks the quantity against the available stock, creates the Order record with Payment Status Pending and Delivery Status Not Started, and shows a confirmation message. An email is sent to the farm manager for confirmation. The rest of the process continues through the internal automation described in Chapter 13.

15.7 Benefits of the Portal

- Farmers get transparency on the sale of their produce and payments.
- Buyers get direct access to available stock and reduce dependency on middlemen.
- Managers spend less time answering phone calls about status and stock.
- All communication and orders are logged in one system, improving trust.

[Insert Screenshot: Experience Builder pages, portal login, farmer and buyer views]

CHAPTER 16: USER MANUAL

16.1 Introduction

This user manual explains how each type of user works with the Farm Management System. It is written in simple steps so that a farmer or field officer with basic computer knowledge can use the application.

16.2 Logging In

- Open the login URL sent to you by email in a web browser or open the Salesforce mobile app.
- Enter your username and password and click Log In.
- If asked, enter the verification code sent to your email or authenticator app.
- After login, select Farm Management System from the App Launcher if it is not already open.
- To log out, click your profile picture at the top right and choose Log Out.

16.3 Navigation

The navigation bar at the top shows the tabs Home, Farmers, Farms, Crops, Harvests, Buyers, Orders, Reports and Dashboards. Click a tab to see a list of records. Use the search box at the top to find any record by name or phone number. Use the plus icon for quick creation of a new record and the bell icon for notifications and approval requests.

16.4 Manual for the Farm Manager

16.4.1 Register a New Farmer and Farm

- On the Home page click Register Farmer and Farm.
- Fill in the farmer details and click Next.
- Fill in the farm details and click Next.
- Check the confirmation message and click the link to open the new record.

16.4.2 Add a Crop to a Farm

- Open the Farm record from the Farms tab.
- In the Crops related list click New.

- Select Crop Name, Season, Variety, Sowing Date, Expected Harvest Date and Expected Yield.
- Click Save. The stage is set to Sowing and a follow-up task is created automatically.

16.4.3 Update the Crop Stage

- Open the Crop record and use the Path at the top of the page.
- Click the stage that applies and choose Mark as Current Stage.
- If the crop is complete, choose Harvested and then record the harvest.

16.4.4 Record a Harvest

- Open the Crop record and click New in the Harvests related list.
- Enter the harvest date, quantity in kilograms, quality grade and storage location.
- Click Save. Available quantity is set automatically.

16.4.5 Create an Order

- Open the Orders tab and click New.
- Select the Buyer and the Harvest, enter the quantity and price per kg and save.
- The total amount is calculated automatically. If the value is high, click Submit for Approval.
- When the buyer pays, edit the order and change Payment Status to Paid. Delivery Status will change to Ready.

16.4.6 View Reports and Dashboards

- Click the Dashboards tab and open Farm Management Dashboard.
- Use the filters at the top to select district or season.
- Click Refresh to update the numbers and click any chart to open the underlying report.

16.5 Manual for the Farmer

- Log in and open My Farms to see your farms and land details.
- Open a farm to see the crops planned or growing on it and the number of days to harvest.
- Open Harvests to check the quantity and available stock of your produce.
- Open Orders to check which buyer has purchased your produce, the price and the payment status.

- If any detail is incorrect, use the Edit button on your own farm or crop to correct it or contact the farm manager.

16.6 Manual for the Buyer

- Log in to Farm Connect and open Available Produce.
- Choose a harvest, note the quality grade and available quantity, and click Place Order.
- Enter the quantity and delivery address and submit the order.
- Track the payment and delivery status in My Orders and use the contact link if you need help.

16.7 Manual for the System Administrator

- Create users from Setup, Users, and assign the correct profile, role and permission sets.
- Reset passwords and unlock users from the user record.
- Monitor flow errors, Apex jobs and login history regularly.
- Take a weekly data export from Setup, Data Export.
- Review reports on data quality, such as farmers without phone numbers or farms without crops.

16.8 Common Problems and Solutions

Problem	Possible Cause	Solution
Cannot save a crop	Harvest date is not after sowing date	Correct the dates
Cannot save an order	Quantity is more than available stock	Reduce the quantity or record a new harvest
Cannot see a farm	Farm belongs to another user or region	Ask the manager to share the record
Reminder email not received	Email is missing or in spam	Update the farmer email and check spam folder
Dashboard shows no data	Filters too restrictive	Clear filters and refresh
Cannot mark order delivered	Payment is not complete	Update the payment first

16.9 Frequently Asked Questions

Can I use the application on my phone?

Yes. Install the Salesforce mobile app from the Play Store or App Store and log in using the same username and password. The layouts adjust automatically for smaller screens.

Can I import many records at once?

Yes. Administrators can use the Data Import Wizard for up to 50,000 records or the Data Loader for larger volumes. Prepare a CSV file with the correct column names.

What happens if I delete a farm?

Because Crop is a master-detail child of Farm, all crops of the farm are also deleted and moved to the Recycle Bin. Records can be restored from the Recycle Bin for 15 days.

Who can see the identity number of a farmer?

Only the System Administrator and Farm Manager profiles have access. It is hidden from all other users.

CHAPTER 17: PROJECT PLAN, DATA MIGRATION AND EXTENDED TEST LOG

17.1 Project Plan and Timeline

Week	Phase	Activities	Deliverable
1	Analysis	Study reference, gather requirements, write user stories	Requirement document
2	Design	Design data model, security and automation	Design document, schema
3	Build I	Create org, objects, fields, relationships, tabs and app	Working data model
4	Build II	Layouts, record pages, path, profiles, roles, sharing	Secure application
5	Build III	Validation rules, flows, approval process	Automated processes
6	Build IV	Apex trigger, batch job, LWC, REST API	Custom code with tests
7	Analytics	Reports, dashboards, sample data	Reports and dashboard
8	Testing	Unit, integration, security and UAT testing	Test report
9	Documentation	Final report, screenshots, demo video	Project report
10	Review	Presentation, viva preparation, submission	Final submission

17.2 Effort and Cost Estimate

Item	Effort in Hours	Remarks
Requirement and design	30	Analysis and documentation
Configuration of objects and security	40	Fields, layouts, profiles
Automation and flows	35	Nine flows and approval
Apex and LWC	30	Trigger, batch, LWC
Reports and dashboard	15	Ten reports and one dashboard
Testing and fixes	30	All test levels
Documentation and demo	20	Report and video
Total	200	About 10 weeks at 20 hours per week

The development cost for the academic project is nil because the Developer Edition is free. For a production deployment, licence costs depend on the Salesforce edition and the number of users, and would be added to the effort cost.

17.3 Data Migration Plan

17.3.1 Sequence of Load

Data must be loaded in dependency order so that lookups can be resolved: Farmer, Farm, Crop, Harvest, Buyer, Order. External IDs, such as Identity Number on Farmer and GST Number on Buyer, are used to link child records to parents in the import.

17.3.2 Sample CSV: Farmers

Name	Phone	Email	Address	Identity Number	Experience Years	Status
Murugan	9876500001	murugan@example.com	Thanjavur	ID1001	15	Active
Lakshmi	9876500002	lakshmi@example.com	Salem	ID1002	8	Active
Selvam	9876500003	selvam@example.com	Erode	ID1003	20	Active
Kavitha	9876500004	kavitha@example.com	Madurai	ID1004	6	Active
Ramesh	9876500005	ramesh@example.com	Coimbatore	ID1005	12	Active

17.3.3 Sample CSV: Farms

Farm Name	Farmer Identity Number	District	Area Acres	Soil Type	Irrigation Type
Green Valley Farm	ID1001	Thanjavur	5.0	Alluvial	Canal
Sunrise Farm	ID1002	Salem	3.5	Red	Borewell
River Bend Farm	ID1003	Erode	8.0	Black	Canal
Hill View Farm	ID1004	Madurai	2.0	Red	Drip
Palm Grove Farm	ID1005	Coimbatore	4.5	Sandy	Sprinkler

17.3.4 Sample CSV: Buyers

Buyer Name	Company	Buyer Type	Phone	GST Number
Kumar Traders	Kumar and Sons	Trader	9123400001	33AAAAA0000A1Z5
Agro Coop Society	Agro Cooperative	Cooperative	9123400002	33BBBBB1111B1Z6
FreshMart	FreshMart Retail	Retailer	9123400003	33CCCCC2222C1Z7
Southern Foods	Southern Foods Pvt Ltd	Processor	9123400004	33DDDDD3333D1Z8

17.3.5 Data Validation After Load

After each load, verify record counts with a report, check that no farm exists without a farmer, no crop without a farm and no order without buyer and harvest, and correct any errors reported in the import result file.

17.4 Extended Test Log

ID	Scenario	Steps	Expected	Actual	Status
TL-01	Login as Farm Manager	Log in and open app	App loads with all tabs	As expected	Pass
TL-02	Login as Farmer	Log in and open Farms tab	Only own farms	As expected	Pass
TL-03	Create Farmer via screen flow	Fill two screens	Farmer and Farm created	As expected	Pass
TL-04	Search by phone number	Enter phone in search	Farmer found	As expected	Pass
TL-05	Import 5 farmers by CSV	Use Data Import Wizard	5 records created	As expected	Pass
TL-06	Edit farm area to 0	Save	Error shown	As expected	Pass
TL-07	Add 3 crops to a farm	Create crops	Total Crops equals 3	As expected	Pass
TL-08	Yield per acre formula	Expected yield 12000, area 5	Value 2400	As expected	Pass
TL-09	Path stage change	Click Growing	Stage updates	As expected	Pass
TL-10	Field history	Change stage and check history	Old and new values logged	As expected	Pass
TL-11	Delete farm	Delete and check crops	Crops also deleted	As expected	Pass
TL-12	Restore from recycle bin	Undelete	Farm and crops restored	As expected	Pass
TL-13	Harvest low stock alert	Reduce stock below 10 percent	Task and notification created	As expected	Pass
TL-14	Buyer credit limit	Order exceeds credit limit	Approval required	As expected	Pass
TL-15	Approval by manager	Approve order	Record unlocked and status updated	As expected	Pass
TL-16	Rejection of order	Reject with comment	Comment stored	As expected	Pass
TL-17	Portal login as buyer	Login to Farm Connect	Available Produce visible	As expected	Pass
TL-18	Portal order placement	Place order via flow	Order created Pending	As expected	Pass
TL-19	Farmer portal restrictions	Try to access other farmer data	Access denied	As expected	Pass
TL-20	REST API call	GET stock endpoint with token	JSON list of stock	As expected	Pass

ID	Scenario	Steps	Expected	Actual	Status
TL-21	LWC summary	Open Farm page	Summary card shows correct numbers	As expected	Pass
TL-22	Mobile view	Open app on phone	Layout responsive	As expected	Pass
TL-23	Report filter	Filter Kharif crops	Only Kharif records	As expected	Pass
TL-24	Dashboard filter	Select district	Charts update	As expected	Pass
TL-25	Bulk insert 200 crops	Load through Apex test	No limit error	As expected	Pass

17.5 Traceability Matrix

Requirement	Implemented By	Test Cases
FR-01 Farmer records	Farmer object, screen flow	TC-01, TC-02, TL-03
FR-02 Farms	Farm object and lookup	TC-03, TC-05, TL-06
FR-03 Crops	Crop object and master-detail	TC-06, TC-08, TL-07
FR-04 Crop stage	Path, flows	TC-15, TC-17, TL-09
FR-05 Harvest	Harvest object, flow	TC-09, TC-18
FR-07 Orders	Order object, formula	TC-19, TC-22
FR-08 Payment and delivery	Flow, validation	TC-13, TC-21
FR-10 Reminder email	Scheduled flow	TC-16
FR-12 Stock control	Apex trigger	TC-19, TC-20
FR-13 Reports	Report types	TC-33, TC-34
FR-14 Dashboard	Dashboard	TC-35, TC-36
FR-15 Security	Profiles, roles, sharing	TC-26 to TC-32

CHAPTER 18: RESULTS, FUTURE SCOPE AND CONCLUSION

18.1 Results Achieved

- A fully functional Salesforce application with six custom objects, six flows, eight validation rules, three Apex classes with a trigger, ten reports and one dashboard.
- Complete traceability from farmer to farm, crop, harvest, order and buyer.
- Automated crop lifecycle, harvest reminders, stock control and payment-based delivery updates.
- Secure access for administrators, managers and farmers with different visibility.
- Real-time insight into production, sales and pending payments.

18.2 Benefits

- Centralized and secure storage of agricultural data.
- Reduction of manual work and human error through automation.
- Better planning of seasons, labour, storage and sales using dashboards.
- Transparent buyer transactions and faster payment follow-up.
- Scalable and low-maintenance solution on the Salesforce cloud.

18.3 Limitations

- The Developer Edition has limited storage, email and API capacity.
- Buyers and farmers depend on managers for some data entry as portal access is not implemented.
- The system does not include real-time weather, market price or sensor data.

18.4 Future Enhancements

- Experience Cloud portal so that farmers and buyers can log in, view records and place orders.
- Salesforce mobile app customization with offline data capture and photo upload of crops.
- Weather API integration to provide rainfall and temperature alerts on farm records.

- Mandi price integration to show daily market prices and suggest the best time to sell.
- Einstein AI or Agentforce to predict yield, detect disease from photos and recommend actions.
- Payment gateway integration for online payment collection.
- Government scheme, insurance and loan tracking objects.
- IoT integration for soil moisture and irrigation sensors using Platform Events.
- Multi-language support (Tamil, Hindi, Telugu) for farmer-friendly usage.

18.5 Conclusion

The Farm Management System successfully demonstrates how the Salesforce platform can be used to digitize agricultural operations. By combining a well-designed data model, strong security, automation through flows and Apex, and insight through reports and dashboards, the system replaces scattered paper records with a reliable, real-time, and secure solution. It supports farmers, farm managers and administrators in making better decisions, reducing losses and improving income. The modular design allows the system to grow with additional features such as portals, mobile access, weather integration and artificial intelligence, making it a strong foundation for smart and sustainable agriculture.

APPENDIX A: FORMULA LIBRARY

Purpose	Object	Formula
Order total	Order__c	Quantity_Kg__c * Price_per_Kg__c
Days to harvest	Crop__c	Expected_Harvest_Date__c - TODAY()
Crop duration in days	Crop__c	Expected_Harvest_Date__c - Sowing_Date__c
Harvest delayed flag	Crop__c	AND(Days_To_Harvest__c < 0, NOT(ISPICKVAL(Stage__c, "Harvested")))
Yield per acre	Farm__c	IF(Area_Acres__c > 0, Total_Expected_Yield__c / Area_Acres__c, 0)
Stock used percent	Harvest__c	IF(Quantity_Kg__c > 0, (Quantity_Kg__c - Available_Quantity__c) / Quantity_Kg__c, 0)
Order age in days	Order__c	TODAY() - Order_Date__c
Payment overdue flag	Order__c	AND(Order_Age__c > 30, NOT(ISPICKVAL(Payment_Status__c, "Paid")))
Farm size category	Farm__c	IF(Area_Acres__c < 2.5, "Marginal", IF(Area_Acres__c < 5, "Small", IF(Area_Acres__c < 10, "Medium", "Large")))
Season from month	Crop__c	IF(AND(MONTH(Sowing_Date__c) >= 6, MONTH(Sowing_Date__c) <= 10), "Kharif", IF(OR(MONTH(Sowing_Date__c) >= 11, MONTH(Sowing_Date__c) <= 3), "Rabi", "Zaid"))
Total with GST 5 percent	Order__c	Total_Amount__c * 1.05

APPENDIX B: SOQL QUERY LIBRARY

```
// All active farmers with their farms
SELECT Name, Phone__c, (SELECT Name, Area_Acres__c FROM Farms__r) FROM Farmer__c WHERE
Status__c = 'Active'
// Crops ready to harvest in the next 7 days
SELECT Name, Crop_Name__c, Farm__r.Name, Expected_Harvest_Date__c
FROM Crop__c WHERE Expected_Harvest_Date__c = NEXT_N_DAYS:7 AND Stage__c != 'Harvested'
// Total farm area by district
SELECT District__c, SUM(Area_Acres__c) totalArea FROM Farm__c GROUP BY District__c
// Revenue by buyer
SELECT Buyer__r.Name, SUM(Total_Amount__c) revenue FROM Order__c GROUP BY Buyer__r.Name ORDER
BY SUM(Total_Amount__c) DESC
// Pending payments older than 30 days
SELECT Name, Buyer__r.Name, Total_Amount__c FROM Order__c
WHERE Payment_Status__c != 'Paid' AND Order_Date__c < LAST_N_DAYS:30
// Harvests with stock remaining
SELECT Name, Crop__r.Crop_Name__c, Available_Quantity__c FROM Harvest__c WHERE
Available_Quantity__c > 0
// Farmers with more than one farm
SELECT Farmer__c, COUNT(Id) FROM Farm__c GROUP BY Farmer__c HAVING COUNT(Id) > 1
// Crop count by season and stage
SELECT Season__c, Stage__c, COUNT(Id) FROM Crop__c GROUP BY Season__c, Stage__c
```

APPENDIX C: GLOSSARY

Term	Meaning
Object	A database table in Salesforce that stores records of one type
Field	A column of an object that stores one piece of information
Record	A single row of data in an object
Lookup	A loose relationship between two objects
Master-Detail	A tight relationship where the child depends on the parent
Roll-Up Summary	A field on the master that calculates count, sum, min or max of child records
Formula Field	A read-only field whose value is calculated from other fields
Validation Rule	A rule that blocks saving of invalid data
Flow	A declarative automation tool for business processes
Apex	Salesforce programming language for server-side logic
Trigger	Apex code that runs before or after a record is created, updated or deleted
SOQL	Query language to retrieve records from Salesforce
Profile	A set of permissions that defines what a user can do
Permission Set	A collection of extra permissions assigned to users
Role	A position in the hierarchy that decides record visibility
OWD	Organization-Wide Defaults, the baseline record access
Governor Limits	Runtime limits that ensure fair use of shared resources
Sandbox	A copy of production used for development and testing
Lightning App Builder	Tool to design apps and pages without code

APPENDIX D: VIVA QUESTIONS AND ANSWERS

Q1. What is the aim of your project?

To build a centralized Salesforce application that records and manages farmer, farm, crop, harvest, buyer and order data securely and provides automation and dashboards for better decision making.

Q2. Why did you choose Salesforce?

Because it offers rapid development with clicks, built-in security, automation, reports, mobile access and scalability without maintaining servers.

Q3. What is the difference between Lookup and Master-Detail?

In a lookup the child can exist independently and has its own security. In master-detail the child depends on the parent, inherits its sharing, is deleted with the parent and allows roll-up summaries. I used master-detail between Farm and Crop.

Q4. What is a validation rule? Give an example from your project.

A validation rule checks data before saving and blocks it if the formula is true. Example: Expected_Harvest_Date__c <= Sowing_Date__c blocks a harvest date earlier than sowing.

Q5. Which automation tools did you use?

Record-triggered flows, scheduled flows, a screen flow, an approval process, email templates and Apex trigger and batch classes.

Q6. What is the difference between Flow and Apex?

Flow is declarative and suitable for most business automation, while Apex is code for complex logic, bulk processing and integrations. I used Flow for reminders and status updates and Apex for stock control and batch alerts.

Q7. How did you secure the data?

By using profiles, roles, permission sets, organization-wide defaults, sharing rules and field-level security so each user sees only what they need.

Q8. What are governor limits?

Limits that Salesforce places on code execution, such as SOQL queries and DML statements per transaction, to protect the multi-tenant environment.

Q9. What is a roll-up summary field?

A field on the master object that calculates values such as count or sum from child records. In the project, Farm shows Total Crops and Total Expected Yield.

Q10. What reports and dashboards did you create?

Reports for farms by district, crops by season and stage, upcoming harvests, sales by buyer, pending payments and monthly sales. A dashboard combines these into one view.

Q11. What is a sandbox?

A separate copy of the org used for development and testing before deployment to production.

Q12. How would you extend the project?

By adding an Experience Cloud portal for farmers and buyers, weather and market price integration, mobile offline support and AI-based yield prediction.

Q13. What is bulkification?

Writing Apex so that it handles many records at once using collections, without SOQL or DML inside loops.

Q14. What is a permission set?

A group of additional permissions assigned to users without changing their profile, following least privilege.

Q15. What is field history tracking?

A feature that records changes to selected fields, including old value, new value, user and time, for auditing.

APPENDIX E: CHECKLIST FOR SUBMISSION

- Screenshots of all objects, fields, relationships and schema builder.
- Screenshots of the Lightning App, record pages and Path.
- Screenshots of profiles, roles, permission sets and sharing settings.
- Screenshots of validation rules, flows and Apex classes.
- Screenshots of reports and the dashboard.
- Sample data screenshots and test case execution results.
- Demo video of the working application and the final report.

APPENDIX F: STUDY OF EXISTING SYSTEMS AND LITERATURE REVIEW

F.1 Introduction

Before designing the Farm Management System, several existing agricultural software solutions and research approaches were studied to understand their strengths and limitations. The study helped to identify the gap that this project aims to fill: a low-code, secure, extensible platform that links farmer data with market and order data in one system.

F.2 Existing Farm Management Solutions

System Type	Examples of Features	Limitations
Spreadsheet based	Manual entry of crops and expenses	No security, no automation, errors and duplication
Mobile advisory apps	Weather, crop advice, price alerts	Little record keeping, no buyer or order tracking
Standalone farm ERP	Inventory, accounting, field records	Costly, complex, difficult to customize
Government portals	Land records, subsidies, insurance	Not integrated with day-to-day farm operations
Marketplace apps	Direct selling to buyers	No crop planning or farm data management

F.3 Research Observations

- Digital record keeping improves planning accuracy and reduces input wastage, since decisions are based on historical data.
- Farmers adopt technology faster when interfaces are simple, mobile friendly and available in local languages.
- Traceability from farm to buyer improves quality assurance and access to premium markets and export channels.
- Automated alerts for sowing, irrigation and harvest reduce losses caused by delays.
- Cloud platforms reduce the cost of ownership because organizations do not maintain their own servers.

F.4 Gap Analysis

Need	Available in Existing Systems	Provided by This Project
Farmer, farm and crop records together	Partly	Yes, with relationships and roll-ups
Harvest to order traceability	Rarely	Yes, complete chain
Role-based data security	Limited	Yes, profiles, roles, sharing, field security
Automation without coding	No	Yes, Flow builder
Custom dashboards	Limited	Yes, drag and drop reports and dashboards
Portal for farmers and buyers	Separate apps	Yes, Experience Cloud
Extensibility with AI and APIs	Difficult	Yes, Apex, REST and Einstein

F.5 Justification of the Proposed System

The proposed system uses the Salesforce platform to provide most of these features through configuration instead of extensive programming. This reduces development time, gives enterprise security and allows future features to be added quickly. The system is therefore suitable both for a student project and as a prototype for a real cooperative or agri-business.

APPENDIX G: DIAGRAM DESCRIPTIONS

G.1 Context Diagram (Level 0 Data Flow)

The Farm Management System is the central process. External entities are the Farmer, the Farm Manager, the Buyer and the Administrator. The Farmer supplies farm and crop details and receives reminders, harvest status and order status. The Farm Manager supplies crop updates, harvest records and orders and receives reports and dashboards. The Buyer supplies order requests and receives confirmations and delivery updates. The Administrator supplies configuration and receives logs and audit information.

G.2 Level 1 Data Flow Diagram

- Process 1.0 Manage Farmers and Farms: reads and writes the Farmer and Farm data stores.
- Process 2.0 Manage Crops: reads Farm and writes Crop, and triggers reminders.
- Process 3.0 Manage Harvest: reads Crop and writes Harvest, updating crop stage.
- Process 4.0 Manage Buyers and Orders: reads Buyer and Harvest, writes Order, and updates stock.
- Process 5.0 Reports and Dashboards: reads all data stores and provides analytics to managers.
- Process 6.0 Security and Administration: manages users, roles, profiles and permission sets.

G.3 Sequence of Order Processing

- The manager or buyer creates an Order record selecting the buyer, harvest, quantity and price.
- The before-insert trigger validates that the quantity does not exceed the available quantity of the harvest.
- The record is saved, the total amount is calculated by the formula, and the after-insert trigger reduces the available quantity of the harvest.
- If the order value is high, the approval process is started and the record is locked until approval.
- When the buyer pays, the payment status is changed to Paid and the flow sets the delivery status to Ready and sends a confirmation email.
- The logistics team delivers the goods and the delivery status is changed to Delivered.
- The reports and dashboard reflect the sale in revenue and pending payment figures.

G.4 Crop Life Cycle State Description

A crop begins in the Sowing state when it is created. After 15 days, a scheduled flow moves it to Growing. When the expected harvest date arrives, the scheduled flow moves it to Ready to Harvest. When a Harvest record is created for the crop, the crop moves to Harvested, which is the final state. Managers can also move the crop between states manually using the Path component if a change of plan is required.

G.5 Component Diagram Description

The presentation layer consists of the Lightning App and the Experience Cloud site. The service layer consists of Flows, Apex classes, triggers, the REST resource and the LWC controller. The data layer consists of the six custom objects. The security layer wraps all layers through profiles, roles, sharing and field-level security. The analytics layer reads from the data layer and displays through reports and dashboards.

G.6 Deployment Description

Users access the application through browsers or the mobile app over HTTPS. The requests are routed by Salesforce to the multi-tenant instance where the org is hosted. Data is stored in Salesforce database with automatic backups, and scheduled jobs run on the Salesforce platform, so no local server is needed by the user.

APPENDIX H: EXTENDED VIVA QUESTIONS

Q16. What is the difference between a profile and a role?

A profile controls what a user can do, such as object and field permissions, while a role controls what records the user can see through the hierarchy. Every user must have one profile, while a role is optional.

Q17. What is the difference between a workflow, Process Builder and Flow?

Workflow rules and Process Builder are older automation tools that are retired for new development. Flow is the current and most powerful tool that combines their features and adds screens, loops and subflows.

Q18. What is a before trigger and an after trigger?

A before trigger runs before the record is saved and is used to validate or update the same record without extra DML. An after trigger runs after the record is saved and is used to work on related records or use the record Id.

Q19. Why did you use an auto number for Crop, Harvest and Order?

An auto number creates a unique, readable identifier such as CROP-0001 that is easy to quote in conversations, reports and emails.

Q20. What is a junction object?

An object with two master-detail relationships used to create a many-to-many relationship. In this project, if a harvest could be sold to many buyers in one combined order, Order Line could be used as a junction object.

Q21. What is the difference between a report and a dashboard?

A report lists and summarizes records based on filters and groupings. A dashboard shows visual components built from reports so that key indicators are visible on one page.

Q22. What are Lightning Web Components?

They are modern web components based on standard web technologies used to build custom user interface elements in Salesforce. In this project, a component shows a farm summary card.

Q23. What is the use of the Schema Builder?

It shows objects and relationships visually and allows creating them by drag and drop. It is useful for documenting the data model.

Q24. What is an external ID?

A custom field marked as external ID that can be used to match records during import and upsert. Identity Number on Farmer is an example.

Q25. How do you avoid duplicate records?

By using matching rules and duplicate rules, unique fields such as phone and identity number, and validation rules.

Q26. What is the difference between with sharing and without sharing in Apex?

With sharing enforces the current user's record access rules, while without sharing ignores them. The project uses with sharing by default for security.

Q27. What is a sandbox and why is it used?

A sandbox is a copy of the production org for development and testing without affecting live data.

Q28. What is a queueable, batch and future method?

They are asynchronous Apex options. Future methods run simple tasks later, Queueable jobs allow chaining and complex objects, and Batch Apex processes large numbers of records in chunks. This project uses Batch for overdue payments.

Q29. How would you protect sensitive data like identity numbers?

By field-level security, restricted profiles, encryption using Shield or encrypted field types, and audit trails.

Q30. How do you test Apex code?

By writing test classes with the isTest annotation, creating test data, running the code between Test.startTest and Test.stopTest, and asserting the results. At least 75 percent coverage is needed for deployment.

Q31. What are the limitations of your project?

It is built in a developer org with limited storage and email limits, does not integrate real-time weather or price data, and depends on internet connectivity for use.

Q32. What would you do to improve adoption among farmers?

Provide local language support, voice input, a simple mobile interface, training sessions and offline capability.

Q33. How does your project support decision making?

By providing dashboards on crop distribution, yield, sales and pending payments, so managers can plan seed, labour, storage and selling times.

Q34. What is the role of formulas in your project?

Formulas calculate values such as order total, days to harvest, yield per acre and balance amount automatically, avoiding manual errors.

Q35. Can this project be used in a real organization?

Yes, with additional licences, production security review, data migration and training. It can be extended for cooperatives, farmer producer organizations and agri-input companies.

APPENDIX I: DEMONSTRATION SCRIPT AND PRESENTATION OUTLINE

I.1 Presentation Outline

- Slide 1: Title, student details and project name.
- Slide 2: Problem statement and need for the system.
- Slide 3: Objectives and scope.
- Slide 4: Solution overview and architecture.
- Slide 5: Data model with the entity relationship diagram.
- Slide 6: Security model.
- Slide 7: Automation using flows and approval.
- Slide 8: Apex trigger, batch and LWC.
- Slide 9: Reports and dashboard.
- Slide 10: Testing summary and results.
- Slide 11: Future enhancements.
- Slide 12: Conclusion and thank you.

I.2 Live Demonstration Script

- Log in as the Farm Manager and show the app navigation.
- Run the Register Farmer and Farm screen flow and open the new records.
- Add a crop with valid dates and show the automatic stage and task, then try invalid dates to show the validation message.
- Create a harvest and show the stock quantity and the crop stage change.
- Create an order, show the total amount and the stock reduction, then mark it as paid and show the delivery status and email.
- Log in as a Farmer and show that only own records are visible and that the identity number is hidden.
- Open the dashboard and explain each component, then apply a district filter.

- Close with the portal view for the buyer.

I.3 Tips for a Successful Presentation

- Keep the sample data realistic and consistent so the story flows naturally from farmer to buyer.
- Practise the demonstration so the flows and emails work on the first attempt.
- Keep screenshots ready as a backup in case the internet connection fails.
- Explain the business benefit of each feature before showing the technical part.
- Prepare short answers to the viva questions in Appendices D and H.

REFERENCES

- Salesforce Trailhead: Data Modeling, Flow Builder, Apex Basics, Data Security and Reports and Dashboards.
- Salesforce Help and Documentation: help.salesforce.com and developer.salesforce.com.
- Project reference document: FARM MANAGEMENT SYSTEM (NM), Google Docs.